

Contents

DP

D&C DP optimization	1
SOS	1

DS

2DSPARSE TABLE	1
Alien Trick	1
BIT	1
Centroid Decom	1
HASHING	2
HLD and Merge sort tree	2
HLD	3
LCA	3
LINE EQ	3
MOs	3
Persistence trie	3
Trie	3
persistentSegment	4
randomnumber	4
xor_basis	4

Flow

Dinic	5
edmondkarp	5
mcmf	5

General

formulas	6
generate_codebook	6
sublime build	7
template	7

Geometry

Convex hull	7
Geo 2	7
Nearest point	7
geo_2d	8

Graph

Articulation Point_Bridge	13
Bell	13
DSU ON Tree	13
DSU	14

euler_path_directed	14
kosaraju	14
mos on trees	14
roll_dsu	15

Number Theory Math

CRT	15
Diophantile	15
FFT	15
LSieveBigModTernaryS	16
Miller Rabin	16
NTT	16
matexpo	16
mobius	17
ncr	17
stirling_number	17
totient	17

String

AhoCorasick	18
Manacher	18
Suffix Automata	18
palindromitree	19
suffixArray	19
z kmp	19

DP

D&C DP optimization

```
1 /**
2  Let opt(i, j) be the value of k that minimizes the above
   expression. Assuming that the cost function satisfies
   the quadrangle inequality, we can show that opt(i, j)
   <= opt(i, j + 1) for all
3  i, j. This is known as the monotonicity condition. Then, we
   can apply divide and conquer DP. The optimal "
   splitting point" for a fixed i increases as j
   increases.
4  */
5  int const N = 3e5 + 5, inf = 1e16;
6  int m, n, R = 0, L = 1, res = 0;
7  int ar[N], cnt[N];
8  vector<long long> dp_before, dp_cur;
9  void add(int id) {
10     res += cnt[ar[id]]; cnt[ar[id]]++;
11 }
12 void Remove(int id) {
13     cnt[ar[id]]--; res -= cnt[ar[id]];
14 }
15 long long C(int l, int r) {
```

```
16     while(R < r) add(++R);
17     while(L > l) add(--L);
18     while(L < l) Remove(L--);
19     while(R > r) Remove(R--);
20     return res;
21 }
22 void compute(int l, int r, int optl, int optr) {
23     if (l > r) return;
24     int mid = (l + r) >> 1;
25     pair<long long, int> best = {LLONG_MAX, -1};
26     for (int k = optl; k <= min(mid, optr); k++) best = min(
27         best, {(k ? dp_before[k - 1] : 0) + C(k, mid), k});
28     dp_cur[mid] = best.first;
29     int opt = best.second;
30     compute(l, mid - 1, optl, opt); compute(mid + 1, r, opt,
31         optr);
32 }
33 void solve() {
34     cin >> n >> m;
35     for (int i = 1; i <= n; i++) cin >> ar[i];
36     dp_before.assign(n + 1, 0); dp_cur.assign(n + 1, 0);
37     for (int i = 1; i <= n; i++) dp_before[i] = C(1, i);
38     for (int i = 1; i < m; i++) {
39         compute(1, n, 1, n); dp_before = dp_cur;
40     }
41     cout << dp_before[n] << endl;
42 }
```

SOS

```
1 const int B = 20;
2 int a[1 << B], f[1 << B], g[1 << B];
3 void solve() {
4     int n; cin >> n;
5     for (int i = 1; i <= n; i++) {
6         cin >> a[i];
7         f[a[i]]++;
8         g[a[i]]++;
9     }
10    // sum over subsets
11    for (int i = 0; i < B; i++) {
12        for (int mask = 0; mask < (1 << B); mask++) {
13            if ((mask & (1 << i)) != 0) {
14                f[mask] += f[mask ^ (1 << i)];
15            }
16        }
17    }
18    // sum over supersets
19    for (int i = 0; i < B; i++) {
20        for (int mask = (1 << B) - 1; mask >= 0; mask--) {
21            if ((mask & (1 << i)) == 0) g[mask] += g[mask ^ (1 <<
22                i)];
23        }
24    }
25    for (int i = 1; i <= n; i++) {
26        cout << f[a[i]] << ' ' << g[a[i]] << ' ' << n - f[(1 <<
27            B) - 1 - a[i]] << '\n';
28    }
```

DS

2DSPARSE TABLE

```

1 const ll N=1000; const ll M=10; ll tab[M+1][M+1][N+1][N
+1]; ll L[N+1]; ll a[N+1][N+1]; ll s[N+1][N+1];
2 struct st2{ ll n,m; void init(ll _n,ll _m){ n=_n; m=_m; for
(ll i=2;i<=N;i++){ L[i]=L[i/2]+1; } } ll g(ll x,ll y,
ll z=0,ll w=0){ return max({x,y,z,w}); } void build(){
for(ll i=0;i<n;i++){ for(ll j=0;j<m;j++){ tab[0][0][i
][j]=a[i][j]; } } for(ll y=1;y<=M;y++){ for(ll i=0;i<n
;i++){ for(ll j=0;j<m;j++){ if(j+(1<<y)-1<m) tab[0][y
][i][j]=g(tab[0][y-1][i][j],tab[0][y-1][i][j+(1<<(y-1
))]); } } } for(ll x=1;x<=M;x++){ for(ll i=0;i<n;i++){
for(ll j=0;j<m;j++){ if(i+(1<<x)-1<n) tab[x][0][i][j]=
g(tab[x-1][0][i][j],tab[x-1][0][i+(1<<(x-1))][j]); } }
for(ll x=1;x<=M;x++){ for(ll y=1;y<=M;y++){ for(ll i=0;i<n;i++){
for(ll j=0;j<m;j++){ if(i+(1<<x)-1<n &&
j+(1<<y)-1<m) { tab[x][y][i][j]=g( tab[x-1][y-1][i][j],
tab[x-1][y-1][i+(1<<(x-1))][j], tab[x-1][y-1][i][j
+(1<<(y-1))], tab[x-1][y-1][i+(1<<(x-1))][j+(1<<(y-1))
]); } } } } } ll qry_i(ll x,ll y,ll _x,ll _y){ ll
lx=L[_x-x+1]; ll ly=L[_y-y+1]; return g( tab[lx][ly][x
][y], tab[lx][ly][_x-(1<<lx)+1][y], tab[lx][ly][x][_y
-(1<<(ly))+1], tab[lx][ly][_x-(1<<lx)+1][_y-(1<<(ly))
+1]); } } };

```

Alien Trick

```

1 // Aliens trick on Tree
2 // Maximizes total value using <= K selections
3 struct State {
4     long long val; // best value
5     int cnt; // number of selections
6 };
7 vector<pair<int,int>> g[N]; // {to, weight}
8 int K;
9 // merge two states
10 State merge(State a, State b) {
11     return {a.val + b.val, a.cnt + b.cnt};
12 }
13 State dfs(int u, int p, long long penalty) {
14     State res = {0, 0};
15     for (auto [v, w] : g[u]) {
16         if (v == p) continue;
17         State cur = dfs(v, u, penalty);
18         // choose edge
19         State take = {cur.val + w - penalty, cur.cnt + 1};
20         // choose better
21         if (take.val > cur.val) res = merge(res, take);
22         else res = merge(res, cur);
23     }
24     return res;
25 }
26 // check if we can do with <= K selections
27 bool check(long long penalty) {
28     State res = dfs(1, 0, penalty);
29     return res.cnt <= K;
30 }
31 // binary search on penalty
32 long long solve() {
33     long long lo = -1e12, hi = 1e12;
34     while (lo < hi) {
35         long long mid = (lo + hi + 1) / 2;

```

```

36         if (check(mid)) lo = mid;
37         else hi = mid - 1;
38     }
39     return dfs(1, 0, lo).val + lo * K;
40 }

```

BIT

```

1 struct FenwickTree {
2     vector<int> bit; int n;
3     FenwickTree(int n) {
4         this->n = n; bit.assign(n, 0);
5     }
6     FenwickTree(vector<int> const &a) : FenwickTree(a.size())
7     {
8         for (size_t i = 0; i < a.size(); i++)
9             add(i, a[i]);
10    }
11    int sum(int r) {
12        int ret = 0;
13        for (; r >= 0; r = (r & (r + 1)) - 1)
14            ret += bit[r];
15        return ret;
16    }
17    int sum(int l, int r) {
18        return sum(r) - sum(l - 1);
19    }
20    void add(int idx, int delta) {
21        for (; idx < n; idx = idx | (idx + 1))
22            bit[idx] += delta;
23    }
24 }

```

Centroid Decom

```

1 struct node {
2     vector<int> adj; int sub = 1; int p = -1; // centroid
3     int dep = 0; int vis = 0; // removed (centroid)
4 };
5 node tree[N]; int n; ll ans = 0; int root;
6 /* ----- SUBTREE SIZE ----- */
7 void dfs(int u, int p) {
8     tree[u].sub = 1;
9     for (int v : tree[u].adj) {
10        if (v != p && tree[v].vis == 0) {
11            dfs(v, u);
12            tree[u].sub += tree[v].sub;
13        }
14    }
15 }
16 /* ----- FIND CENTROID ----- */
17 int dfs2(int u, int p, int tot) {
18     for (int v : tree[u].adj) {
19         if (v != p && tree[v].vis == 0 && tree[v].sub > tot /
20             2) {
21             return dfs2(v, u, tot);
22         }
23     }
24     return u;
25 }
26 /* ----- COLLECT LEVEL INFO (OPTIONAL) ----- */
27 /* Keep this if you need depth / distance info */

```

```

27 void nextlev(int u, int p, int d) {
28     tree[u].dep = d;
29     for (int v : tree[u].adj) {
30         if (v != p && tree[v].vis == 0) {
31             nextlev(v, u, d + 1);
32         }
33     }
34 }
35 /* ----- SOLVE FOR A CENTROID ----- */
36 /*
37 // void solve(int c) {
38 // // example placeholder // do calculations using centroid
39 // }
40 // ----- BUILD DECOMPOSITION ----- */
41 void build(int u, int p) {
42     dfs(u, -1);
43     int c = dfs2(u, -1, tree[u].sub);
44     tree[c].p = p;
45     if (p == -1) root = c;
46     // optional: gather depth info from centroid
47     nextlev(c, -1, 0);
48     solve(c);
49     tree[c].vis = 1;
50     for (int v : tree[c].adj) {
51         if (tree[v].vis == 0) {
52             build(v, c);
53         }
54     }
55 }
56 // Build the tree inside main and call build(1, -1);

```

HASHING

```

1 int power(long long n, long long k, const int mod) { int
ans = 1 % mod; n %= mod; if (n < 0) n += mod; while (k
) { if (k & 1) ans = (long long) ans * n % mod; n =
(long long) n * n % mod; k >>= 1; } return ans; } const
int MOD1 = 127657753, MOD2 = 987654319; const int p1
= 137, p2 = 277; int ip1, ip2; pair<int, int> pw[N],
ipw[N]; void prec() { pw[0] = {1, 1}; for (int i = 1;
i < N; i++) { pw[i].first = 1LL * pw[i - 1].first * p1
% MOD1; pw[i].second = 1LL * pw[i - 1].second * p2 %
MOD2; } ip1 = power(p1, MOD1 - 2, MOD1); ip2 = power(
p2, MOD2 - 2, MOD2); ipw[0] = {1, 1}; for (int i = 1;
i < N; i++) { ipw[i].first = 1LL * ipw[i - 1].first *
ip1 % MOD1; ipw[i].second = 1LL * ipw[i - 1].second *
ip2 % MOD2; } } struct Hashing { int n; string s; // 0
- indexed vector<pair<int, int>> hs; // 1 - indexed
// Hashing() {} void init(string _s) { n = _s.size();
s = _s; hs.emplace_back(0, 0); for (int i = 0; i < n;
i++) { pair<int, int> p; p.first = (hs[i].first + 1LL
* pw[i].first * s[i] % MOD1) % MOD1; p.second = (hs[i
].second + 1LL * pw[i].second * s[i] % MOD2) % MOD2;
hs.push_back(p); } } pair<int, int> get_hash(int l,
int r) { // 1 - indexed assert(1 <= l && l <= r && r
<= n); pair<int, int> ans; ans.first = (hs[r].first -
hs[l - 1].first + MOD1) * 1LL * ipw[l - 1].first %
MOD1; ans.second = (hs[r].second - hs[l - 1].second +
MOD2) * 1LL * ipw[l - 1].second % MOD2; return ans; }
pair<int, int> get_hash() { return get_hash(1, n); }
};

```

HLD and Merge sort tree

```

1 typedef struct node {
2     int p;
3     vector<int> adj;
4     int mxchild; int chain; int ind_in_chain; int subtree;
5     int koto; int price;
6 } node;
7 node tree[200010];
8 vector<int> chain(200010);
9 vector<int> start_c(200010);
10 vector<vector<int>> seg(800010);
11 vector<vector<ll>> sum(800010);
12 vector<int> dfs_array(200010);
13 // vector<vector<seg>> segtree(500010);
14 int c;
15 int ind=-1;
16 void dfs(int u, int p) {
17     tree[u].p = p;
18     int mx=0;
19     f(i, tree[u].adj.size()) {
20         int v = tree[u].adj[i];
21         if(p==v) continue;
22         dfs(v,u);
23         tree[u].subtree += tree[v].subtree;
24         if(mx < tree[v].subtree) {mx = tree[v].subtree; tree[
25             u].mxchild = v;}
26     }
27 void hld(int u) {
28     dfs_array[++ind] = u;
29     chain[c]++;
30     // chain[c].push_back(u);
31     tree[u].chain = c;
32     tree[u].ind_in_chain = chain[c] - 1;
33     if(tree[u].mxchild) hld(tree[u].mxchild);
34     f(i, tree[u].adj.size()) {
35         int v = tree[u].adj[i];
36         if(v == tree[u].p || v == tree[u].mxchild) continue;
37         c++;
38         start_c[c] = ind+1;
39         hld(v);
40     }
41 }
42 void merge(int ind) {
43     int lsize = seg[2*ind+1].size();
44     int rsize = seg[2*ind+2].size();
45     seg[ind].clear();
46     sum[ind].clear();
47     seg[ind].resize(lsize+rsize);
48     sum[ind].resize(lsize+rsize);
49     int l = 0, r = 0, i=0;
50     while(l < lsize || r < rsize) {
51         if(l==lsize) {
52             seg[ind][i] = (seg[2*ind+2][r]);r++;
53         }
54         else if(r==rsize) {
55             seg[ind][i] = (seg[2*ind+1][l]);l++;
56         }
57         else if(seg[2*ind+1][l] < seg[2*ind+2][r]) {
58             seg[ind][i] = (seg[2*ind+1][l]);l++;
59         }
60         else {
61             seg[ind][i] = (seg[2*ind+2][r]);r++;
62         }

```

```

63         sum[ind][i] = ((l==0 ? 0 : sum[ind*2+1][l-1]) + (r==0
64             ? 0 : sum[ind*2+2][r-1]));
65         i++;
66     }
67     // return narr;
68 void build(vector<int>& arr, int l, int r, int ind) {
69     if(l==r) {
70         l = arr[l];
71         seg[ind].resize(1);
72         sum[ind].resize(1);
73         seg[ind][0] = (tree[l].price);
74         sum[ind][0] = ((ll)tree[l].koto);
75     } else {
76         int m = (l+r)/2;
77         build(arr,l,m,2*ind+1); build(arr,m+1,r,2*ind+2);
78         merge(ind);
79     }
80 }
81 ll query(int s, int e, int l, int r, int ind, int tar) {
82     if(s>e) return 0;
83     if(s==l && e==r) {
84         int ans = -1;
85         l = 0, r = seg[ind].size()-1;
86         while(l <= r) {
87             int m = (l+r)/2;
88             if(seg[ind][m] <= tar)
89                 ans = m, l = m+1;
90             else r = m-1;
91         }
92         return ans==-1?0:sum[ind][ans];
93     } else {
94         int m = (l+r)/2;
95         return query(s,min(e,m),l,m,2*ind+1,tar) + query(max(
96             s,m+1),e,m+1,r,2*ind+2,tar);
97     }
98 }

```

HLD

```

1 vector<int> parent, depth, heavy, head, pos;
2 int cur_pos;
3 int dfs(int v, vector<vector<int>> const& adj) {
4     int size = 1; int max_c_size = 0;
5     for (int c : adj[v]) {
6         if (c != parent[v]) {
7             parent[c] = v, depth[c] = depth[v] + 1;
8             int c_size = dfs(c, adj); size += c_size;
9             if (c_size > max_c_size)
10                 max_c_size = c_size, heavy[v] = c;
11         }
12     }
13     return size;
14 }
15 void decompose(int v, int h, vector<vector<int>> const& adj) {
16     head[v] = h, pos[v] = cur_pos++;
17     if (heavy[v] != -1) decompose(heavy[v], h, adj);
18     for (int c : adj[v]) {
19         if (c != parent[v] && c != heavy[v]) decompose(c, c,
20             adj);
21     }
22 void init(vector<vector<int>> const& adj) {

```

```

23     int n = adj.size(); parent = vector<int>(n); depth =
24         vector<int>(n);
25     heavy = vector<int>(n, -1); head = vector<int>(n);
26     pos = vector<int>(n); cur_pos = 0;
27     dfs(0, adj);
28     decompose(0, 0, adj);
29 }

```

LCA

```

1 typedef struct node {
2     int val;vector<int> adj;vector<int> p;vector<int> ele;
3     int childv;int dep;
4 } node;
5 node tree[100005];
6 void dfs(int u, int p) {
7     tree[u].p[0] = p;
8     tree[u].dep = tree[p].dep + 1;
9     for(int i = 1; i <= (int)log2(n) + 1; i++) {
10         if(tree[u].p[i-1] == 0) break;
11         tree[u].p[i] = tree[tree[u].p[i-1]].p[i-1];
12     }
13     for(int i = 0; i < tree[u].adj.size(); i++) if(tree[u].
14         adj[i] != p) dfs(tree[u].adj[i],u);
15 }
16 int lca(int u, int v) {
17     if(tree[v].dep < tree[u].dep) swap(v,u);
18     for(int i = (int)log2(n) + 1; i >= 0; i--) {
19         if(tree[v].p[i] && (tree[tree[v].p[i]].dep >= tree[u].
20             dep)) {
21             v = tree[v].p[i];
22         }
23     }
24     if(v==u) return v;
25     for(int i = (int)log2(n) + 1; i >= 0; i--) {
26         if(tree[v].p[i] != tree[u].p[i]) {
27             v = tree[v].p[i]; u = tree[u].p[i];
28         }
29     }
30     return tree[v].p[0];
31 }

```

LINE EQ

```

1 //all solution of ax+by=c in [minx,maxx], [miny,maxy]
2 int gcd(int a, int b, int &x, int &y) { if (b == 0) { x =
3     1; y = 0; return a; } int x1, y1; int d = gcd(b, a % b
4     , x1, y1); x = y1; y = x1 - y1 * (a / b); return d; }
5 bool solve_any(int a, int b, int c, int &x, int &y,
6     int &g) { if (a == 0 && b == 0) { g = x = y = 0;
7     return true; } g = gcd(abs(a), abs(b), x, y); if (c %
8     g != 0) { return false; } x *= (c / g); y *= (c / g);
9     if (a < 0) x = -x; if (b < 0) y = -y; return true; }
10 void shift(int &x, int &y, int a, int b, int cnt) { x
11     += cnt * b; y -= cnt * a; } int solve_all(int a, int b
12     , int c, int minx, int maxx, int miny, int maxy) { int
13     x, y, g; if (!solve_any(a, b, c, x, y, g)) return 0;
14     a /= g; b /= g; int sign_a = a > 0 ? +1 : -1; int
15     sign_b = b > 0 ? +1 : -1; shift(x, y, a, b, (minx - x)
16     / b); if (x < minx) { shift(x, y, a, b, sign_b); } if
17     (x > maxx) return 0; int lxl = x; shift(x, y, a, b, (
18     maxx - x) / b); if (x > maxx) { shift(x, y, a, b, -
19     sign_b); } int rxl = x; shift(x, y, a, b, (y - miny) /

```

```

a); if (y < miny) { shift(x, y, a, b, -sign_a); } if
(y > maxy) return 0; int lx2 = x; shift(x, y, a, b, (y
- maxy) / a); if (y > maxy) { shift(x, y, a, b,
sign_a); } int rx2 = x; if (lx2 > rx2) swap(lx2, rx2);
int lx = max(lx1, lx2); int rx = min(rx1, rx2); if (
lx > rx) return 0; return (rx - lx) / abs(b) + 1; }
void solve() { int a, b, c; cin >> a >> b >> c; int
minx, miny, maxx, maxy; cin >> minx >> maxx >> miny >>
maxy; if (maxx < minx || maxy < miny) { cout << 0 <<
endl; exit(0); } if (a == 0 && b == 0) { if (c == 0) {
cout << (maxy - miny + 1) * (maxx - minx + 1) << endl;
} else { cout << 0 << endl; } } else if (a == 0 || b
== 0) { if (a == 0) { if (c % b == 0 && (c / b) >=
miny && (c / b) <= maxy) { cout << maxx - minx + 1 <<
endl; } else { cout << 0 << endl; } } else { if (c % a
== 0 && (c / a) >= minx && (c / a) <= maxx) { cout <<
maxy - miny + 1 << endl; } else { cout << 0 << endl;
} } } else cout << solve_all(a, b, c, minx, maxx, miny
, maxy) << endl; }

```

MOs

```

1 const int BLOCK_SIZE = 350;
2 int n, q, ar[N], freq[N], res = 0, ans[10*N];
3 struct Query {
4     int l, r, id;
5     bool operator<(const Query &y) const {
6         int x_block = l / BLOCK_SIZE;
7         int y_block = y.l / BLOCK_SIZE;
8         if (x_block == y_block)
9             return r < y.r;
10        return x_block < y_block;
11    }
12 };
13 void add(int i) {
14     int x = ar[i]; res -= freq[x] / 2; freq[x]++; res +=
        freq[x] / 2;
15 }
16 void Remove(int i) {
17     int x = ar[i]; res -= freq[x] / 2; freq[x]--; res +=
        freq[x] / 2;
18 }
19 void solve() { // need to reset freq if needed
20     cin >> n;
21     for (int i = 1; i <= n; i++) cin >> ar[i];
22     cin >> q;
23     vector<Query> qr;
24     res = 0; int ii = 0;
25     while (q--) {
26         int l, r; cin >> l >> r; qr.push_back(Query({l, r, ii
            ++}));
27     }
28     sort(qr.begin(), qr.end());
29     int left = 1, right = 0;
30     for (Query q : qr) {
31         int l = q.l, r = q.r;
32         while (right < r) add(++right);
33         while (left > l) add(--left);
34         while (left < l) Remove(left++);
35     }
36     while (right > r) Remove(right--);
37     ans[q.id] = res;
38 }
39 for (int i = 0; i < ii; i++) cout << ans[i] << "\n";
40 }

```

Persistence trie

```

1 map<ll, ll> root; struct node { ll left = 0, right = 0; ll
    cnt = 0; }; node a[9000001]; ll ptr=2; struct p_trie{
    p_trie() { root[0] = 1; } void insert(ll par_id, ll
    curr_id, ll k) { root[curr_id]=ptr++; ll par_node=root[
    par_id]; ll curr_node=root[curr_id]; for (ll i=30; i>=0;
    i--) { if ((k << i)) { a[curr_node].left = a[
    par_node].left; a[curr_node].right = ptr++; curr_node
    = a[curr_node].right; par_node = a[par_node].right; }
    else { a[curr_node].right = a[par_node].right; a[
    curr_node].left = ptr++; curr_node = a[curr_node].left;
    par_node = a[par_node].left; } a[curr_node].cnt = a[
    par_node].cnt + 1; } } ll max_xor(ll id, ll k) { ll curr=
    root[id]; ll ans=0; for (ll i=30; i>=0; i--) { if ((k << i))
    { if (a[curr].left) { ans |= (1ll << i); curr = a[curr].
    left; } else { curr = a[curr].right; } } else { if (a[curr].
    right) { ans |= (1ll << i); curr = a[curr].right; } else { curr
    = a[curr].left; } } } return ans; } ll min_xor(ll id, ll
    k) { ll curr=root[id]; ll ans=0; for (ll i=30; i>=0; i--)
    { if ((k << i)) { if (a[curr].right) { curr = a[curr].
    right; } else { ans |= (1ll << i); curr = a[curr].left; } }
    else { if (a[curr].left) { curr = a[curr].left; } else { ans
    |= (1ll << i); curr = a[curr].right; } } } return ans; } };

```

Trie

```

1 const int N = 3e5 + 9;
2 struct Trie {
3     static const int B = 31;
4     struct node {
5         node* nxt[2];
6         int sz;
7         node() {
8             nxt[0] = nxt[1] = NULL;
9             sz = 0;
10        }
11    } *root;
12    Trie() {
13        root = new node();
14    }
15    void insert(int val) {
16        node* cur = root;
17        cur->sz++;
18        for (int i = B - 1; i >= 0; i--) {
19            int b = val >> i & 1;
20            if (cur->nxt[b] == NULL) cur->nxt[b] = new node();
21            cur = cur->nxt[b];
22            cur->sz++;
23        }
24    }
25    int query(int x, int k) { // number of values s.t. val ^
        x < k
26        node* cur = root;
27        int ans = 0;
28        for (int i = B - 1; i >= 0; i--) {
29            if (cur == NULL) break;
30            int b1 = x >> i & 1, b2 = k >> i & 1;
31            if (b2 == 1) {
32                if (cur->nxt[b1]) ans += cur->nxt[b1]->sz;
33                cur = cur->nxt[b1];
34            } else cur = cur->nxt[b1];

```

```

35    }
36    return ans;
37 }
38 int get_max(int x) { // returns maximum of val ^ x
39     node* cur = root;
40     int ans = 0;
41     for (int i = B - 1; i >= 0; i--) {
42         int k = x >> i & 1;
43         if (cur->nxt[!k]) cur = cur->nxt[!k], ans <= 1,
            ans++;
44         else cur = cur->nxt[k], ans <= 1;
45     }
46     return ans;
47 }
48 int get_min(int x) { // returns minimum of val ^ x
49     node* cur = root;
50     int ans = 0;
51     for (int i = B - 1; i >= 0; i--) {
52         int k = x >> i & 1;
53         if (cur->nxt[k]) cur = cur->nxt[k], ans <= 1;
54         else cur = cur->nxt[!k], ans <= 1, ans++;
55     }
56     return ans;
57 }
58 void del(node* cur) {
59     for (int i = 0; i < 2; i++) if (cur->nxt[i]) del(cur
        ->nxt[i]);
60     delete(cur);
61 }
62 } t;

```

persistentSegment

```

1 struct node {
2     ll sum; node *left, *right;
3     node(ll val = 0) {
4         sum = val; left = right = NULL;
5     }
6     node(node *l, node *r) {
7         left = l; right = r; sum = (l ? l->sum : 0) + (r ? r
            ->sum : 0);
8     }
9 };
10 int n; vector<int> v;
11 node* build(int start, int end) {
12     if (start == end)
13         return new node(v[start]);
14     int mid = (start + end) >> 1;
15     return new node(build(start, mid), build(mid + 1, end));
16 }
17 node* update(node* prev, int pos, ll val, int start, int
    end) {
18     if (start == end) return new node(val);
19     int mid = (start + end) >> 1;
20     if (pos <= mid) return new node(update(prev->left, pos,
        val, start, mid), prev->right);
21     else return new node(prev->left, update(prev->right, pos
        , val, mid + 1, end));
22 }
23 ll query(node* cur, int l, int r, int start, int end) {
24     if (cur == NULL || start > r || end < l) return 0;
25     if (l <= start && end <= r) return cur->sum;
26     int mid = (start + end) >> 1;
27     return query(cur->left, l, r, start, mid) + query(cur->
        right, l, r, mid + 1, end);

```

```

28 }
29 void solve() {
30     cin >> n; v.resize(n);
31     for (int &x : v) cin >> x;
32     vector<node*> roots; roots.push_back(build(0, n - 1));
33     int q; cin >> q;
34     while (q--) {
35         int id; cin >> id;
36         if (id == 1) {
37             int ver, pos; ll val; cin >> ver >> pos >> val;
38             roots.push_back(update(roots[ver], pos - 1, val,
39                                     0, n - 1));
39         } else {
40             int ver, l, r; cin >> ver >> l >> r;
41             cout << query(roots[ver], l - 1, r - 1, 0, n - 1)
42                  << "\n";
43         }
44     }

```

randomnumber

```

1 struct custom_hash {
2     static uint64_t splitmix64(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6         return x ^ (x >> 31);
7     }
8     size_t operator()(uint64_t x) const {
9         static const uint64_t FIXED_RANDOM = chrono::
10 steady_clock::now().time_since_epoch().count();
11         return splitmix64(x + FIXED_RANDOM);}; //For long long int
        to long long int

```

xor_basis

```

1 struct XorBasis {
2     vector<ll> basis;
3     ll N = 0, tmp = 0;
4     void add(ll x) {
5         N++;
6         tmp ^= x;
7         for (ll b : basis) x = min(x, x ^ b);
8         if (!x) return;
9         for (ll &b : basis) if ((b ^ x) < b) b ^= x;
10        basis.push_back(x);
11        sort(basis.rbegin(), basis.rend());
12    }
13    // number of basis vectors (rank)
14    ll size() { return (ll) basis.size(); }
15    // clear everything
16    void clear() { N = 0; tmp = 0; basis.clear(); }
17    // returns true if x can be formed
18    bool possible(ll x) {
19        for (ll b : basis) x = min(x, x ^ b);
20        return x == 0;
21    }
22    // returns maximum xor value
23    ll maxxor(ll x = 0) {
24        for (ll b : basis) x = max(x, x ^ b);
25        return x;
26    }

```

```

27 // returns minimum xor value
28 ll minxor(ll x = 0) {
29     for (ll b : basis) x = min(x, x ^ b);
30     return x;
31 }
32 // returns count of subsets which xor to x (modded)
33 ll cntxor(ll x) {
34     if (!possible(x)) return 0LL;
35     ll freebits = N - size();
36     ll res = 1;
37     while (freebits--) res = (res * 2) % MOD;
38     return res;
39 }
40 // returns sum of xor of all subsets
41 ll sumOfAll() {
42     return tmp * (1LL << (N - 1));
43 }
44 // returns k-th smallest xor value (1-indexed)
45 ll kth(ll k) {
46     ll sz = size();
47     if (k > (1LL << sz)) return -1;
48     k--;
49     ll ans = 0;
50     for (ll i = 0; i < sz; i++)
51         if (k >> (sz - 1 - i) & 1) ans ^= basis[i];
52     return ans;
53 }
54 } xB;

```

Flow Dinic

```

1 struct FlowEdge {
2     int v, u;
3     int cap, flow = 0;
4     FlowEdge(int v, int u, int cap) : v(v), u(u), cap(cap) {}
5 };
6 struct Dinic {
7     const int flow_inf = 1e8;
8     vector<FlowEdge> edges;
9     vector<vector<int>> adj;
10    int n, m = 0;
11    int s, t;
12    vector<int> level, ptr;
13    queue<int> q;
14    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
15        adj.resize(n);
16        level.resize(n);
17        ptr.resize(n);
18    }
19    void add_edge(int v, int u, int cap) {
20        edges.emplace_back(v, u, cap);
21        edges.emplace_back(u, v, cap); // cap if undirected,
22        otherwise 0;
23        adj[v].push_back(m);
24        adj[u].push_back(m + 1);
25        m += 2;
26    }
27    bool bfs() {
28        while (!q.empty()) {
29            int v = q.front();
30            q.pop();

```

```

31        for (int id : adj[v]) {
32            if (edges[id].cap == edges[id].flow)
33                continue;
34            if (level[edges[id].u] != -1)
35                continue;
36            level[edges[id].u] = level[v] + 1;
37            q.push(edges[id].u);
38        }
39        return level[t] != -1;
40    }
41    int dfs(int v, int pushed) {
42        if (pushed == 0) return 0;
43        if (v == t) return pushed;
44        for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
45            int id = adj[v][cid];
46            int u = edges[id].u;
47            if (level[v] + 1 != level[u]) continue;
48            int tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
49            if (tr == 0) continue;
50            edges[id].flow += tr; edges[id ^ 1].flow -= tr;
51            return tr;
52        }
53        return 0;
54    }
55    int flow() {
56        int f = 0;
57        while (true) {
58            fill(level.begin(), level.end(), -1);
59            level[s] = 0;
60            q.push(s);
61            if (!bfs()) break;
62            fill(ptr.begin(), ptr.end(), 0);
63            while (int pushed = dfs(s, flow_inf)) {
64                f += pushed;
65            }
66        }
67        return f;
68    }
69 }
70 vector<int> min_cut() {
71     vector<int> vis(n, 0);
72     queue<int> q;
73     q.push(s);
74     vis[s] = 1;
75     while (!q.empty()) {
76         int v = q.front();
77         q.pop();
78         for (int id : adj[v]) {
79             int u = edges[id].u;
80             if (!vis[u] && edges[id].cap > edges[id].flow) {
81                 vis[u] = 1;
82                 q.push(u);
83             }
84         }
85     }
86     return vis; // vis[v] = 1 S-side of cut
87 }
88 };

```

edmondkarp

```

1 // edmonds karp ve^2 call while(bfs(n) != 0)
2 typedef struct node {
3     int vis; int p; vector<int> adj; vector<ll> cap;
4 } node;
5 node graph[519];
6 ll bfs(int n) {
7     queue<int> qu;
8     vector<bool> vis(510,0);
9     vis[1] = 1;
10    qu.push(1);
11    while(!qu.empty()) {
12        int cur = qu.front();
13        // printf("cur-- %d\n", cur);
14        qu.pop();
15        f(i, graph[cur].adj.size()) {
16            int nex = graph[cur].adj[i];
17            if(vis[nex] == 0 && graph[cur].cap[nex] > 0) {
18                graph[nex].p = cur; vis[nex] = 1;
19                if(nex == n) break;
20                qu.push(nex);
21            }
22        }
23    }
24    if(vis[n] == 0) return 0;
25    ll maxflow = LLONG_MAX;
26    int cur = n;
27    while(cur != 1) {
28        maxflow = min(maxflow, graph[graph[cur].p].cap[cur]);
29        cur = graph[cur].p;
30        // printf("%d ", cur);
31    }
32    // printf("max %d\n", maxflow);
33    cur = n;
34    while(cur != 1) {
35        graph[graph[cur].p].cap[cur] -= maxflow;
36        graph[cur].cap[graph[cur].p] += maxflow;
37        cur = graph[cur].p;
38    }
39    return maxflow;
40 }
41
42 // call after getting max flow
43 vector<int> getCut(int n) {
44     vector<int> vis(510,0);
45     queue<int> q;
46     q.push(1);
47     vis[1] = 1;
48     while(!q.empty()) {
49         int u = q.front(); q.pop();
50         for(int v : graph[u].adj) {
51             if(!vis[v] && graph[u].cap[v] > 0) { // residual
52                 capacity
53                 vis[v] = 1;
54                 q.push(v);
55             }
56         }
57     }
58     return vis; // vis[v] = 1 -> reachable -> S-side

```

mcmf

```
1 struct Edge
```

```

2 {
3     int from, to, capacity, cost;
4 };
5 vector<vector<int>> adj, cost, capacity;
6 const int INF = 1e9;
7 void shortest_paths(int n, int v0, vector<int>& d, vector<
8     int>& p) {
9     d.assign(n, INF);
10    d[v0] = 0;
11    vector<bool> inq(n, false);
12    queue<int> q;
13    q.push(v0);
14    p.assign(n, -1);
15    while (!q.empty()) {
16        int u = q.front();
17        q.pop();
18        inq[u] = false;
19        for (int v : adj[u]) {
20            if (capacity[u][v] > 0 && d[v] > d[u] + cost[u][v]
21                ) {
22                d[v] = d[u] + cost[u][v];
23                p[v] = u;
24                if (!inq[v]) {
25                    inq[v] = true;
26                    q.push(v);
27                }
28            }
29        }
30    }
31 }
32 int min_cost_flow(int N, vector<Edge> edges, int K, int s,
33     int t) {
34     adj.assign(N, vector<int>());
35     cost.assign(N, vector<int>(N, 0));
36     capacity.assign(N, vector<int>(N, 0));
37     for (Edge e : edges) {
38         adj[e.from].push_back(e.to);
39         adj[e.to].push_back(e.from);
40         cost[e.from][e.to] = e.cost;
41         cost[e.to][e.from] = -e.cost;
42         capacity[e.from][e.to] = e.capacity;
43     }
44     int flow = 0;
45     int cost = 0;
46     vector<int> d, p;
47     while (flow < K) {
48         shortest_paths(N, s, d, p);
49         if (d[t] == INF)
50             break;
51         // find max flow on that path
52         int f = K - flow;
53         int cur = t;
54         while (cur != s) {
55             f = min(f, capacity[p[cur]][cur]);
56             cur = p[cur];
57         }
58         // apply flow
59         flow += f;
60         cost += f * d[t];
61         cur = t;
62         while (cur != s) {
63             capacity[p[cur]][cur] -= f;
64             capacity[cur][p[cur]] += f;
65             cur = p[cur];
66         }
67     }
68 }

```

```

64 }
65 if (flow < K)
66     return -1;
67 else
68     return cost;
69 }

```

General formulas

```

1 /* ----- POLYGON -----
2 Area of polygon (shoelace):
3 Area = 0.5 * |sum(x[i]*y[i+1] - x[i+1]*y[i])|
4 Centroid of polygon:
5 Cx = sum((xi + xi+1) * cross(i,i+1)) / (6*Area)
6 Cy = sum((yi + yi+1) * cross(i,i+1)) / (6*Area)
7 Point inside polygon (ray casting):
8 Count intersections of ray to +x
9 Odd = inside, Even = outside
10 Convex polygon point inside:
11 For all edges (Pi, Pi+1):
12 cross(Pi+1-Pi, X-Pi) >= 0 (or <=0 consistently)
13 */
14 /* ----- TRIANGLE -----
15 Area using cross:
16 Area = |cross(B-A, C-A)| / 2
17 Circumcenter:
18 Intersection of perpendicular bisectors
19 (Omitted formula compute via lines)
20 Incenter:
21 I = (a*A + b*B + c*C) / (a+b+c)
22 where:
23 Centroid:
24 G = (A+B+C)/3
25 Orthocenter:
26 H = A + B + C - 2*O (O = circumcenter)
27 Radius:
28 Inradius r = Area / s
29 Circumradius R = (a*b*c) / (4*Area)
30 Angle between vectors:
31 cos(theta) = dot(u,v)/(|u||v|)
32 */

```

generate_codebook

```

1 import os
2 import subprocess
3 import sys
4
5 ALLOWED_EXTENSIONS = {
6     ".cpp": "C++",
7     ".cc": "C++",
8     ".h": "C++",
9     ".hpp": "C++",
10    ".py": "Python",
11    ".java": "Java",
12 }
13
14 IGNORE_DIRS = {".git", ".github", "build", "__pycache__"}
15
16

```

```

17 def escape_latex(text: str) -> str:
18     replacements = {
19         "\\": r"\textbackslash{}",
20         "_": r"\_",
21         "%": r"\%",
22         "$": r"\$",
23         "#": r"\#",
24         "&": r"\&",
25         "{": r"\{}",
26         "}": r"\}",
27         "~": r"\textasciitilde{}",
28         "^": r"\textasciicircum{}",
29     }
30     for char, rep in replacements.items():
31         text = text.replace(char, rep)
32     return text
33
34 def collect_files():
35     tree = {}
36     for root, dirs, files in os.walk("."):
37         dirs[:] = [d for d in dirs if d not in IGNORE_DIRS
38                     and not d.startswith(".")]
39         rel_dir = os.path.relpath(root, ".")
40         valid_files = [
41             f for f in sorted(files) if os.path.splitext(f)[1]
42             in ALLOWED_EXTENSIONS
43         ]
44         if valid_files:
45             section_name = "General" if rel_dir == "." else
46                 rel_dir
47             tree[section_name] = [
48                 os.path.join(root, f).replace("\\", "/") for f
49                 in valid_files
50             ]
51         return tree
52
53 def build_latex_document(tree, is_color: bool) -> str:
54     color_definitions = (
55         r"""
56 \definecolor{kwcolor}{rgb}{0.0, 0.2, 0.8}
57 \definecolor{cmtcolor}{rgb}{0.0, 0.5, 0.0}
58 \definecolor{strcolor}{rgb}{0.7, 0.1, 0.1}
59 \definecolor{numcolor}{rgb}{0.4, 0.4, 0.4}
60 \definecolor{framecolor}{rgb}{0.7, 0.7, 0.7}
61 """
62         if is_color
63         else r"""
64 \definecolor{kwcolor}{rgb}{0.0, 0.0, 0.0}
65 \definecolor{cmtcolor}{rgb}{0.2, 0.2, 0.2}
66 \definecolor{strcolor}{rgb}{0.0, 0.0, 0.0}
67 \definecolor{numcolor}{rgb}{0.3, 0.3, 0.3}
68 \definecolor{framecolor}{rgb}{0.0, 0.0, 0.0}
69 """
70     )
71     doc = [
72         r"\documentclass[9pt,landscape,a4paper]{article}",
73         r"\usepackage[left=0.8cm,right=0.8cm,top=1.2cm,bottom
74             =1.2cm]{geometry}",
75         r"\usepackage{multicol}",
76         r"\usepackage{listings}",
77         r"\usepackage{xcolor}",
78         r"\usepackage{fancyhdr}",

```

```

77         r"\usepackage{titlesec}",
78         r"\usepackage{tocloft}",
79         r"\usepackage{courier}",
80         color_definitions,
81         r"\lstset{",
82             r"    basicstyle=\ttfamily\scriptsize,",
83             r"    keywordstyle=\bfseries\color{kwcolor},",
84             r"    commentstyle=\itshape\color{cmtcolor},",
85             r"    stringstyle=\color{strcolor},",
86             r"    numberstyle=\tiny\color{numcolor},",
87             r"    numbers=left,",
88             r"    stepnumber=1,",
89             r"    numbersep=4pt,",
90             r"    frame=single,",
91             r"    rulecolor=\color{framecolor},",
92             r"    tabsize=2,",
93             r"    breaklines=true,",
94             r"    breakatwhitespace=false,",
95             r"    showstringspaces=false,",
96             r"    columns=fullflexible,",
97             r"    xleftmargin=12pt",
98         r"}",
99         r"\pagestyle{fancy}",
100         r"\fancyhf{}",
101         r"\fancyhead[L]{\textbf{Algorithmic Code Library}}",
102         (
103             r"\fancyhead[C]{\textit{Color Edition}}"
104             if is_color
105             else r"\fancyhead[C]{\textit{B\&W Edition}}"
106         ),
107         r"\fancyhead[R]{\thepage}",
108         r"\renewcommand{\headrulewidth}{0.4pt}",
109         r"\titlespacing*{\section}{0pt}{4pt}{2pt}",
110         r"\titlespacing*{\subsection}{0pt}{3pt}{1pt}",
111         r"\begin{document}",
112         r"\begin{multicols*}{3}",
113         r"\tableofcontents",
114         r"\vspace{10pt}",
115         r"\hrule",
116         r"\vspace{10pt}",
117     ]
118
119     for section, files in sorted(tree.items()):
120         doc.append(f"\section*{{{escape_latex(section)}}}")
121         doc.append(f"\addcontentsline{toc}{{{section}}}{{{
122             escape_latex(section)}}}")
123
124     for file_path in files:
125         file_name = os.path.basename(file_path)
126         title = os.path.splitext(file_name)[0]
127         ext = os.path.splitext(file_name)[1]
128         lang = ALLOWED_EXTENSIONS.get(ext, "C++")
129
130         doc.append(f"\subsection*{{{escape_latex(title)}}}")
131         doc.append(
132             f"\addcontentsline{toc}{{{subsection}}}{{{
133                 escape_latex(title)}}}"
134         )
135         doc.append(f"\lstinputlisting[language={lang}]{{{
136             file_path}}}")
137
138     doc.append(r"\end{multicols*}")
139     doc.append(r"\end{document}")
140     return "\n".join(doc)

```

```

138
139
140 def compile_latex(tex_name: str):
141     # Run twice so table of contents page numbers compute
142     # correctly
143     for _ in range(2):
144         cmd = ["pdflatex", "-interaction=nonstopmode",
145             tex_name]
146         # We capture the output but don't immediately crash
147         # on a non-zero return code
148         subprocess.run(cmd, stdout=subprocess.PIPE, stderr=
149             subprocess.PIPE)
150
151     # Instead of checking the strict return code, check if
152     # the PDF actually generated
153     pdf_name = tex_name.replace(".tex", ".pdf")
154     if not os.path.exists(pdf_name):
155         print(f"Error: {pdf_name} was not generated!")
156         sys.exit(1)
157
158 def main():
159     tree = collect_files()
160     if not tree:
161         print("No source code files found to compile.")
162         sys.exit(1)
163
164     # 1. Color Edition
165     with open("codebook_color.tex", "w", encoding="utf-8")
166         as f:
167         f.write(build_latex_document(tree, is_color=True))
168         compile_latex("codebook_color.tex")
169
170     # 2. Black & White Edition
171     with open("codebook_bw.tex", "w", encoding="utf-8") as f:
172         f.write(build_latex_document(tree, is_color=False))
173         compile_latex("codebook_bw.tex")
174
175     print("Successfully built codebook_color.pdf and
176         codebook_bw.pdf")
177
178 if __name__ == "__main__":
179     main()

```

sublime build

```

1 {
2     "cmd": ["g++ -std=c++20 $file_name -o $file_base_name &&
3         timeout 5s ./ $file_base_name<input.txt>output.txt"
4     ],
5     "selector": "source.cpp",
6     "shell": true
7     // "working_dir": "$file_path"
8 }

```

template

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define endl '\n'
4 #include <ext/pb_ds/assoc_container.hpp>

```

```

5 #include <ext/pb_ds/tree_policy.hpp>
6 using namespace __gnu_pbds;
7 #define int long long
8 typedef tree<int, null_type, less<int>, rb_tree_tag,
9   tree_order_statistics_node_update> o_set;
10 typedef tree<int, null_type, less_equal<int>, rb_tree_tag,
11   tree_order_statistics_node_update> o_multiset;
12 mt19937 rng(chrono::steady_clock::now().time_since_epoch().
13   count());
14 void solve() { //order_of_key, find_by_order }
15 int32_t main() {
16   ios::sync_with_stdio(false); cin.tie(nullptr);
17   int tc = 1; cin >> tc;
18   for(int i = 1; i <= tc; i++) { solve(); } return 0;
19 }

```

Geometry

Convex hull

```

1 vector<PT> convex_hull(vector<PT> &p) {
2   if (p.size() <= 1) return p;
3   vector<PT> v = p;
4   sort(v.begin(), v.end());
5   vector<PT> up, dn;
6   for (auto& p : v) {
7     while (up.size() > 1 && orientation(up[up.size() -
8       2], up.back(), p) >= 0) {
9       up.pop_back();
10    }
11    while (dn.size() > 1 && orientation(dn[dn.size() -
12      2], dn.back(), p) <= 0) {
13      dn.pop_back();
14    }
15    up.push_back(p);
16    dn.push_back(p);
17  }
18  v = dn;
19  if (v.size() > 1) v.pop_back();
20  reverse(v.begin(), v.end());
21  up.pop_back();
22  for (auto& p : up) {
23    v.push_back(p);
24  }
25  //checks if convex or not
26  bool is_convex(vector<PT> &p) {
27    bool s[3]; s[0] = s[1] = s[2] = 0;
28    int n = p.size();
29    for (int i = 0; i < n; i++) {
30      int j = (i + 1) % n;
31      int k = (j + 1) % n;
32      s[sign(cross(p[j] - p[i], p[k] - p[i])) + 1] = 1;
33      if (s[0] && s[2]) return 0;
34    }
35    return 1;
36  }
37 }

```

```

1 struct point { int x, y, idx; }; //Finds squared euclidean
2 distance between two points int dist(point &a, point &
3 b) { return (a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-b.y);
4 } //Checks if angle ABC is a right angle int
5 isOrthogonal(point &a, point &b, point &c) { return (b
6 .x-a.x) * (b.x-c.x) + (b.y-a.y) * (b.y-c.y) == 0; } //
7 Checks if ABCD form a rectangle (in that order) int
8 isRectangle(point &a, point &b, point &c, point &d) {
9 return isOrthogonal(a, b, c) && isOrthogonal(b, c, d)
10 && isOrthogonal(c, d, a); } //Checks if ABCD form a
11 rectangle, in any orientation int isRectangleAnyOrder(
12 point &a, point &b, point &c, point &d) { return
13 isRectangle(a, b, c, d) || isRectangle(b, c, a, d) |
14 isRectangle(c, a, b, d); } //Checks if ABCD form a
15 square (in that order) int isSquare(point &a, point &b
16 , point &c, point &d) { return isRectangle(a, b, c, d)
17 && dist(a, b) == dist(b, c); } //Checks if ABCD form
18 a square, in any orientation int isSquareAnyOrder(
19 point &a, point &b, point &c, point &d) { return
20 isSquare(a, b, c, d) || isSquare(b, c, a, d) |
21 isSquare(c, a, b, d); }

```

Nearest point

```

1 const int N = 1<<17; const long long INF =
2 2000000000000000000; struct vp_node { long long
3 threshold; pair< int, int > center; vp_node *left, *
4 right; }; typedef vp_node *node_ptr; int tests, n; pair
5 < int, int > pts[N], arr[N]; long long ans; node_ptr
6 root; long long squared_distance(pair< int, int > a,
7 pair< int, int > b) { return (a.first-b.first)*1ll*(a
8 .first-b.first) + (a.second-b.second)*1ll*(a.second-b
9 .second); } struct cmp_points { pair< int, int > a;
10 cmp_points() {} cmp_points(pair< int, int > p): a(p)
11 {} bool operator () (const pair< int, int > &x, const
12 pair< int, int > &y) const { return squared_distance
13 (a,x)<squared_distance(a,y); } }; void build(node_ptr
14 &node, int a, int b) { if(a>b) { node=NULL; return; }
15 node=new vp_node(); if(a==b) { node->threshold=0; node
16 ->center=arr[a]; node->left=NULL; node->right=NULL;
17 return; } int pos=a+rand()% (b-a+1); swap(arr[a], arr[
18 pos]); node->center=arr[pos]; sort(arr+a+1, arr+b+1,
19 cmp_points(arr[a])); node->threshold=squared_distance(
20 arr[a], arr[(a+b)>>1]); build(node->left, a, (a+b)>>1);
21 build(node->right, ((a+b)>>1)+1, b); } void query(
22 node_ptr &node, pair< int, int > q, long long &ans) {
23 if(node==NULL) return; if(node->center!=q) ans=min(
24 ans, squared_distance(node->center, q)); if(node->left==
25 NULL && node->right==NULL) { return; } if(
26 squared_distance(q, node->center) <= node->threshold) {
27 query(node->left, q, ans); if(sqrt(squared_distance(node
28 ->center, q))+sqrt(ans)>sqrt(node->threshold)) query(
29 node->right, q, ans); } else { query(node->right, q, ans);
30 if(sqrt(squared_distance(node->center, q))-sqrt(ans)<
31 sqrt(node->threshold)) query(node->left, q, ans); } }
32 int main() { int i; scanf("%d", &tests); while(tests
33 --) { scanf("%d", &n); for(i=1; i<=n; i++) { scanf("%d %
34 d", &arr[i].first, &arr[i].second); pts[i]=arr[i]; }
35 build(root, 1, n); for(i=1; i<=n; i++) { ans=INF; query(
36 root, pts[i], ans); printf("%lld\n", ans); } } return 0;
37 }

```

geo_2d

```

1 const double PI = acos((double)-1.0); int sign(double x) {
2 return (x > eps) - (x < -eps); } struct PT { double x,
3 y; PT() { x = 0, y = 0; } PT(double x, double y) : x(
4 x), y(y) {} PT(const PT &p) : x(p.x), y(p.y) {} PT
5 operator + (const PT &a) const { return PT(x + a.x, y
6 + a.y); } PT operator - (const PT &a) const { return
7 PT(x - a.x, y - a.y); } PT operator * (const double a)
8 const { return PT(x * a, y * a); } friend PT operator
9 * (const double &a, const PT &b) { return PT(a * b.x,
10 a * b.y); } PT operator / (const double a) const {
11 return PT(x / a, y / a); } bool operator == (PT a)
12 const { return sign(a.x - x) == 0 && sign(a.y - y) ==
13 0; } bool operator != (PT a) const { return !(*this ==
14 a); } bool operator < (PT a) const { return sign(a.x
15 - x) == 0 ? y < a.y : x < a.x; } bool operator > (PT a)
16 const { return sign(a.x - x) == 0 ? y > a.y : x > a.
17 x; } double norm() { return sqrt(x * x + y * y); }
18 double norm2() { return x * x + y * y; } PT perp() {
19 return PT(-y, x); } double arg() { return atan2(y, x);
20 } PT truncate(double r) { // returns a vector with
21 norm r and having same direction double k = norm(); if
22 (!sign(k)) return *this; r /= k; return PT(x * r, y *
23 r); } };
24 istream &operator >> (istream &in, PT &p) { return in >> p.
25 x >> p.y; }
26 ostream &operator << (ostream &out, PT &p) { return out <<
27 "(" << p.x << ", " << p.y << ")"; }
28 inline double dot(PT a, PT b) { return a.x * b.x + a.y * b.
29 y; }
30 inline double dist2(PT a, PT b) { return dot(a - b, a - b);
31 }
32 inline double dist(PT a, PT b) { return sqrt(dot(a - b, a -
33 b)); }
34 inline double cross(PT a, PT b) { return a.x * b.y - a.y *
35 b.x; }
36 inline double cross2(PT a, PT b, PT c) { return cross(b - a
37 , c - a); }
38 inline int orientation(PT a, PT b, PT c) { return sign(
39 cross(b - a, c - a)); }
40 PT perp(PT a) { return PT(-a.y, a.x); }
41 PT rotateccw90(PT a) { return PT(-a.y, a.x); }
42 PT rotatecw90(PT a) { return PT(a.y, -a.x); }
43 PT rotateccw(PT a, double t) { return PT(a.x * cos(t) - a.y
44 * sin(t), a.x * sin(t) + a.y * cos(t)); }
45 PT rotatecw(PT a, double t) { return PT(a.x * cos(t) + a.y
46 * sin(t), -a.x * sin(t) + a.y * cos(t)); }
47 double SQ(double x) { return x * x; }
48 double rad_to_deg(double r) { return (r * 180.0 / PI); }
49 double deg_to_rad(double d) { return (d * PI / 180.0); }
50 double get_angle(PT a, PT b) {
51 double costheta = dot(a, b) / a.norm() / b.norm();
52 return acos(max((double)-1.0, min((double)1.0, costheta)
53 )); }
54 }
55 bool is_point_in_angle(PT b, PT a, PT c, PT p) { // does
56 point p lie in angle <bac
57 assert(orientation(a, b, c) != 0);
58 if (orientation(a, c, b) < 0) swap(b, c);
59 return orientation(a, c, p) >= 0 && orientation(a, b, p)
60 <= 0;
61 }
62 bool half(PT p) {
63 return p.y > 0.0 || (p.y == 0.0 && p.x < 0.0);
64 }

```

Geo 2


```

30 void polar_sort(vector<PT> &v) { // sort points in
    counterclockwise
31     sort(v.begin(), v.end(), [](PT a, PT b) {
32         return make_tuple(half(a), 0.0, a.norm2()) <
            make_tuple(half(b), cross(a, b), b.norm2());
33     });
34 }
35 void polar_sort(vector<PT> &v, PT o) { // sort points in
    counterclockwise with respect to point o
36     sort(v.begin(), v.end(), [&](PT a, PT b) {
37         return make_tuple(half(a - o), 0.0, (a - o).norm2())
            < make_tuple(half(b - o), cross(a - o, b - o), (
                b - o).norm2());
38     });
39 }
40 struct line {
41     PT a, b; // goes through points a and b
42     PT v; double c; // line form: direction vec [cross] (x, y
        ) = c
43     line() {}
44     // direction vector v and offset c
45     line(PT v, double c) : v(v), c(c) {
46         auto p = get_points();
47         a = p.first; b = p.second;
48     }
49     // equation ax + by + c = 0
50     line(double _a, double _b, double _c) : v({_b, -_a}), c(-
        _c) {
51         auto p = get_points();
52         a = p.first; b = p.second;
53     }
54     // goes through points p and q
55     line(PT p, PT q) : v(q - p), c(cross(v, p)), a(p), b(q)
        {}
56     pair<PT, PT> get_points() { // extract any two points
        from this line
57     PT p, q; double a = -v.y, b = v.x; // ax + by = c
58     if (sign(a) == 0) {
59         p = PT(0, c / b); q = PT(1, c / b);
60     }
61     else if (sign(b) == 0) {
62         p = PT(c / a, 0); q = PT(c / a, 1);
63     }
64     else {
65         p = PT(0, c / b); q = PT(1, (c - a) / b);
66     }
67     return {p, q};
68 }
69 // ax + by + c = 0
70 array<double, 3> get_abc() {
71     double a = -v.y, b = v.x; return {a, b, -c};
72 }
73 // 1 if on the left, -1 if on the right, 0 if on the
    line
74 int side(PT p) { return sign(cross(v, p) - c); }
75 // line that is perpendicular to this and goes through
    point p
76 line perpendicular_through(PT p) { return {p, p + perp(v
        )}; }
77 // translate the line by vector t i.e. shifting it by
    vector t
78 line translate(PT t) { return {v, c + cross(v, t)}; }
79 // compare two points by their orthogonal projection on
    this line
80 // a projection point comes before another if it comes
    first according to vector v
81 bool cmp_by_projection(PT p, PT q) { return dot(v, p) <
    dot(v, q); }
82 line shift_left(double d) {
83     PT z = v.perp().truncate(d);
84     return line(a + z, b + z);
85 }
86 };
87 // find a point from a through b with distance d
88 PT point_along_line(PT a, PT b, double d) {
89     assert(a != b);
90     return a + ((b - a) / (b - a).norm()) * d;
91 }
92 // projection point c onto line through a and b assuming a
    != b
93 PT project_from_point_to_line(PT a, PT b, PT c) {
94     return a + (b - a) * dot(c - a, b - a) / (b - a).norm2()
        ;
95 }
96 // reflection point c onto line through a and b assuming a
    != b
97 PT reflection_from_point_to_line(PT a, PT b, PT c) {
98     PT p = project_from_point_to_line(a, b, c);
99     return p + p - c;
100 }
101 // minimum distance from point c to line through a and b
102 double dist_from_point_to_line(PT a, PT b, PT c) {
103     return fabs(cross(b - a, c - a) / (b - a).norm());
104 }
105 // returns true if point p is on line segment ab
106 bool is_point_on_seg(PT a, PT b, PT p) {
107     if (fabs(cross(p - b, a - b)) < eps) {
108         if (p.x < min(a.x, b.x) - eps || p.x > max(a.x, b.x)
            + eps) return false;
109         if (p.y < min(a.y, b.y) - eps || p.y > max(a.y, b.y)
            + eps) return false;
110         return true;
111     }
112     return false;
113 }
114 // minimum distance point from point c to segment ab that
    lies on segment ab
115 PT project_from_point_to_seg(PT a, PT b, PT c) {
116     double r = dist2(a, b);
117     if (sign(r) == 0) return a;
118     r = dot(c - a, b - a) / r;
119     if (r < 0) return a;
120     if (r > 1) return b;
121     return a + (b - a) * r;
122 }
123 // minimum distance from point c to segment ab
124 double dist_from_point_to_seg(PT a, PT b, PT c) {
125     return dist(c, project_from_point_to_seg(a, b, c));
126 }
127 // 0 if not parallel, 1 if parallel, 2 if collinear
128 int is_parallel(PT a, PT b, PT c, PT d) {
129     double k = fabs(cross(b - a, d - c));
130     if (k < eps) {
131         if (fabs(cross(a - b, a - c)) < eps && fabs(cross(c -
            d, c - a)) < eps) return 2;
132         else return 1;
133     }
134     else return 0;
135 }
136 // check if two lines are same
137 bool are_lines_same(PT a, PT b, PT c, PT d) {
138     if (fabs(cross(a - c, c - d)) < eps && fabs(cross(b - c,
        c - d)) < eps) return true;
139     return false;
140 }
141 // bisector vector of <abc
142 PT angle_bisector(PT &a, PT &b, PT &c) {
143     PT p = a - b, q = c - b;
144     return p + q * sqrt(dot(p, p) / dot(q, q));
145 }
146 // 1 if point is ccw to the line, 2 if point is cw to the
    line, 3 if point is on the line
147 int point_line_relation(PT a, PT b, PT p) {
148     int c = sign(cross(p - a, b - a));
149     if (c < 0) return 1;
150     if (c > 0) return 2;
151     return 3;
152 }
153 // intersection point between ab and cd assuming unique
    intersection exists
154 bool line_line_intersection(PT a, PT b, PT c, PT d, PT &ans
        ) {
155     double a1 = a.y - b.y, b1 = b.x - a.x, c1 = cross(a, b);
156     double a2 = c.y - d.y, b2 = d.x - c.x, c2 = cross(c, d);
157     double det = a1 * b2 - a2 * b1;
158     if (det == 0) return 0;
159     ans = PT((b1 * c2 - b2 * c1) / det, (c1 * a2 - a1 * c2)
        / det);
160     return 1;
161 }
162 // intersection point between segment ab and segment cd
    assuming unique intersection exists
163 bool seg_seg_intersection(PT a, PT b, PT c, PT d, PT &ans)
        {
164     double oa = cross2(c, d, a), ob = cross2(c, d, b);
165     double oc = cross2(a, b, c), od = cross2(a, b, d);
166     if (oa * ob < 0 && oc * od < 0) {
167         ans = (a * ob - b * oa) / (ob - oa);
168         return 1;
169     }
170     else return 0;
171 }
172 // intersection point between segment ab and segment cd
    assuming unique intersection may not exists
173 // se.size()==0 means no intersection
174 // se.size()==1 means one intersection
175 // se.size()==2 means range intersection
176 set<PT> seg_seg_intersection_inside(PT a, PT b, PT c, PT d)
        {
177     PT ans;
178     if (seg_seg_intersection(a, b, c, d, ans)) return {ans};
179     set<PT> se;
180     if (is_point_on_seg(c, d, a)) se.insert(a);
181     if (is_point_on_seg(c, d, b)) se.insert(b);
182     if (is_point_on_seg(a, b, c)) se.insert(c);
183     if (is_point_on_seg(a, b, d)) se.insert(d);
184     return se;
185 }
186 // intersection between segment ab and line cd
187 // 0 if do not intersect, 1 if proper intersect, 2 if
    segment intersect
188 int seg_line_relation(PT a, PT b, PT c, PT d) {
189     double p = cross2(c, d, a);
190     double q = cross2(c, d, b);

```

```

191     if (sign(p) == 0 && sign(q) == 0) return 2;
192     else if (p * q < 0) return 1;
193     else return 0;
194 }
195 // intersection between segment ab and line cd assuming
196 // unique intersection exists
197 bool seg_line_intersection(PT a, PT b, PT c, PT d, PT &ans)
198 {
199     bool k = seg_line_relation(a, b, c, d);
200     assert(k != 2);
201     if (k) line_line_intersection(a, b, c, d, ans);
202     return k;
203 }
204 // minimum distance from segment ab to segment cd
205 double dist_from_seg_to_seg(PT a, PT b, PT c, PT d) {
206     PT dummy;
207     if (seg_seg_intersection(a, b, c, d, dummy)) return 0.0;
208     else return min({dist_from_point_to_seg(a, b, c),
209                     dist_from_point_to_seg(a, b, d),
210                     dist_from_point_to_seg(c, d, a),
211                     dist_from_point_to_seg(c, d, b)});
212 }
213 // minimum distance from point c to ray (starting point a
214 // and direction vector b)
215 double dist_from_point_to_ray(PT a, PT b, PT c) {
216     b = a + b;
217     double r = dot(c - a, b - a);
218     if (r < 0.0) return dist(c, a);
219     return dist_from_point_to_line(a, b, c);
220 }
221 // starting point as and direction vector ad
222 bool ray_ray_intersection(PT as, PT ad, PT bs, PT bd) {
223     double dx = bs.x - as.x, dy = bs.y - as.y;
224     double det = bd.x * ad.y - bd.y * ad.x;
225     if (fabs(det) < eps) return 0;
226     double u = (dy * bd.x - dx * bd.y) / det;
227     double v = (dy * ad.x - dx * ad.y) / det;
228     if (sign(u) >= 0 && sign(v) >= 0) return 1;
229     else return 0;
230 }
231 // ray-ray distance
232 double ray_ray_distance(PT as, PT ad, PT bs, PT bd) {
233     if (ray_ray_intersection(as, ad, bs, bd)) return 0.0;
234     double ans = dist_from_point_to_ray(as, ad, bs);
235     ans = min(ans, dist_from_point_to_ray(bs, bd, as));
236     return ans;
237 }
238 struct circle {
239     PT p; double r;
240     circle() {}
241     circle(PT _p, double _r): p(_p), r(_r) {}
242     // center (x, y) and radius r
243     circle(double x, double y, double _r): p(PT(x, y)), r(_r) {}
244     // circumcircle of a triangle
245     // the three points must be unique
246     circle(PT a, PT b, PT c) {
247         b = (a + b) * 0.5;
248         c = (a + c) * 0.5;
249         line_line_intersection(b, b + rotatecw90(a - b), c, c
250                               + rotatecw90(a - c), p);
251         r = dist(a, p);
252     }
253     // inscribed circle of a triangle
254     // pass a bool just to differentiate from circumcircle
255     circle(PT a, PT b, PT c, bool t) {
256         line u, v;
257         double m = atan2(b.y - a.y, b.x - a.x), n = atan2(c.y
258               - a.y, c.x - a.x);
259         u.a = a;
260         u.b = u.a + (PT(cos((n + m)/2.0), sin((n + m)/2.0)));
261         v.a = b;
262         m = atan2(a.y - b.y, a.x - b.x), n = atan2(c.y - b.y,
263               c.x - b.x);
264         v.b = v.a + (PT(cos((n + m)/2.0), sin((n + m)/2.0)));
265         line_line_intersection(u.a, u.b, v.a, v.b, p);
266         r = dist_from_point_to_seg(a, b, p);
267     }
268     bool operator == (circle v) { return p == v.p && sign(r
269           - v.r) == 0; }
270     double area() { return PI * r * r; }
271     double circumference() { return 2.0 * PI * r; }
272 };
273 // 0 if outside, 1 if on circumference, 2 if inside circle
274 int circle_point_relation(PT p, double r, PT b) {
275     double d = dist(p, b);
276     if (sign(d - r) < 0) return 2;
277     if (sign(d - r) == 0) return 1;
278     return 0;
279 }
280 // 0 if outside, 1 if on circumference, 2 if inside circle
281 int circle_line_relation(PT p, double r, PT a, PT b) {
282     double d = dist_from_point_to_line(a, b, p);
283     if (sign(d - r) < 0) return 2;
284     if (sign(d - r) == 0) return 1;
285     return 0;
286 }
287 // compute intersection of line through points a and b with
288 // circle centered at c with radius r > 0
289 vector<PT> circle_line_intersection(PT c, double r, PT a,
290     PT b) {
291     vector<PT> ret;
292     b = b - a; a = a - c;
293     double A = dot(b, b), B = dot(a, b);
294     double C = dot(a, a) - r * r, D = B * B - A * C;
295     if (D < -eps) return ret;
296     ret.push_back(c + a + b * (-B + sqrt(D + eps)) / A);
297     if (D > eps) ret.push_back(c + a + b * (-B - sqrt(D)) /
298           A);
299     return ret;
300 }
301 // 5 - outside and do not intersect
302 // 4 - intersect outside in one point
303 // 3 - intersect in 2 points
304 // 2 - intersect inside in one point
305 // 1 - inside and do not intersect
306 int circle_circle_relation(PT a, double r, PT b, double R)
307 {
308     double d = dist(a, b);
309     if (sign(d - r - R) > 0) return 5;
310     if (sign(d - r - R) == 0) return 4;
311     double l = fabs(r - R);
312     if (sign(d - r - R) < 0 && sign(d - l) > 0) return 3;
313     if (sign(d - l) == 0) return 2;
314     if (sign(d - l) < 0) return 1;
315     assert(0); return -1;
316 }
317 vector<PT> circle_circle_intersection(PT a, double r, PT b,
318     double R) {
319     if (a == b && sign(r - R) == 0) return {PT(1e18, 1e18)};
320     vector<PT> ret;
321     double d = sqrt(dist2(a, b));
322     if (d > r + R || d + min(r, R) < max(r, R)) return ret;
323     double x = (d * d - R * R + r * r) / (2 * d);
324     double y = sqrt(r * r - x * x);
325     PT v = (b - a) / d;
326     ret.push_back(a + v * x + rotateccw90(v) * y);
327     if (y > 0) ret.push_back(a + v * x - rotateccw90(v) * y)
328     ;
329     return ret;
330 }
331 // returns two circle c1, c2 through points a, b and of
332 // radius r
333 // 0 if there is no such circle, 1 if one circle, 2 if two
334 // circle
335 int get_circle(PT a, PT b, double r, circle &c1, circle &c2)
336 {
337     vector<PT> v = circle_circle_intersection(a, r, b, r);
338     int t = v.size();
339     if (!t) return 0;
340     c1.p = v[0], c1.r = r;
341     if (t == 2) c2.p = v[1], c2.r = r;
342     return t;
343 }
344 // returns two circle c1, c2 which is tangent to line u,
345 // goes through
346 // point q and has radius r1; 0 for no circle, 1 if c1 = c2
347 // , 2 if c1 != c2
348 int get_circle(line u, PT q, double r1, circle &c1, circle
349     &c2) {
350     double d = dist_from_point_to_line(u.a, u.b, q);
351     if (sign(d - r1 * 2.0) > 0) return 0;
352     if (sign(d) == 0) {
353         cout << u.v.x << ' ' << u.v.y << '\n';
354         c1.p = q + rotateccw90(u.v).truncate(r1);
355         c2.p = q + rotatecw90(u.v).truncate(r1);
356         c1.r = c2.r = r1;
357         return 2;
358     }
359     line u1 = line(u.a + rotateccw90(u.v).truncate(r1), u.b
360           + rotateccw90(u.v).truncate(r1));
361     line u2 = line(u.a + rotatecw90(u.v).truncate(r1), u.b +
362           rotatecw90(u.v).truncate(r1));
363     circle cc = circle(q, r1);
364     PT p1, p2; vector<PT> v;
365     v = circle_line_intersection(q, r1, u1.a, u1.b);
366     if (!v.size()) v = circle_line_intersection(q, r1, u2.a,
367           u2.b);
368     v.push_back(v[0]);
369     p1 = v[0], p2 = v[1];
370     c1 = circle(p1, r1);
371     if (p1 == p2) {
372         c2 = c1;
373         return 1;
374     }
375     c2 = circle(p2, r1);
376     return 2;
377 }
378 // returns area of intersection between two circles
379 double circle_circle_area(PT a, double r1, PT b, double r2)
380 {
381     double d = (a - b).norm();
382     if (r1 + r2 < d + eps) return 0;
383     if (r1 + d < r2 + eps) return PI * r1 * r1;
384     if (r2 + d < r1 + eps) return PI * r2 * r2;

```

```

361 double theta_1 = acos((r1 * r1 + d * d - r2 * r2) / (2 *
    r1 * d)),
362 theta_2 = acos((r2 * r2 + d * d - r1 * r1)/(2 * r2 * d
    ));
363 return r1 * r1 * (theta_1 - sin(2 * theta_1)/2.) + r2 *
    r2 * (theta_2 - sin(2 * theta_2)/2.);
364 }
365 // tangent lines from point q to the circle
366 int tangent_lines_from_point(PT p, double r, PT q, line &u,
    line &v) {
367     int x = sign(dist2(p, q) - r * r);
368     if (x < 0) return 0; // point in circle
369     if (x == 0) { // point on circle
370         u = line(q, q + rotateccw90(q - p));
371         v = u;
372         return 1;
373     }
374     double d = dist(p, q);
375     double l = r * r / d;
376     double h = sqrt(r * r - l * l);
377     u = line(q, p + ((q - p).truncate(l) + (rotateccw90(q -
        p).truncate(h))));
378     v = line(q, p + ((q - p).truncate(l) + (rotatecw90(q - p
        ).truncate(h))));
379     return 2;
380 }
381 // returns outer tangents line of two circles
382 // if inner == 1 it returns inner tangent lines
383 int tangents_lines_from_circle(PT c1, double r1, PT c2,
    double r2, bool inner, line &u, line &v) {
384     if (inner) r2 = -r2;
385     PT d = c2 - c1;
386     double dr = r1 - r2, d2 = d.norm2(), h2 = d2 - dr * dr;
387     if (d2 == 0 || h2 < 0) {
388         assert(h2 != 0);
389         return 0;
390     }
391     vector<pair<PT, PT>>out;
392     for (int tmp: {-1, 1}) {
393         PT v = (d * dr + rotateccw90(d) * sqrt(h2) * tmp) /
            d2;
394         out.push_back({c1 + v * r1, c2 + v * r2});
395     }
396     u = line(out[0].first, out[0].second);
397     if (out.size() == 2) v = line(out[1].first, out[1].
        second);
398     return 1 + (h2 > 0);
399 }
400
401 double area_of_triangle(PT a, PT b, PT c) {
402     return fabs(cross(b - a, c - a) * 0.5);
403 }
404
405 // -1 if strictly inside, 0 if on the polygon, 1 if
    strictly outside
406 int is_point_in_triangle(PT a, PT b, PT c, PT p) {
407     if (sign(cross(b - a, c - a)) < 0) swap(b, c);
408     int c1 = sign(cross(b - a, p - a));
409     int c2 = sign(cross(c - b, p - b));
410     int c3 = sign(cross(a - c, p - c));
411     if (c1 < 0 || c2 < 0 || c3 < 0) return 1;
412     if (c1 + c2 + c3 != 3) return 0;
413     return -1;
414 }
415 double perimeter(vector<PT> &p) {
416     double ans=0; int n = p.size();
417     for (int i = 0; i < n; i++) ans += dist(p[i], p[(i + 1)
        % n]);
418     return ans;
419 }
420 double area(vector<PT> &p) {
421     double ans = 0; int n = p.size();
422     for (int i = 0; i < n; i++) ans += cross(p[i], p[(i + 1)
        % n]);
423     return fabs(ans) * 0.5;
424 }
425 // centroid of a (possibly non-convex) polygon,
426 // assuming that the coordinates are listed in a clockwise
    or
427 // counterclockwise fashion. Note that the centroid is
    often known as
428 // the "center of gravity" or "center of mass".
429 PT centroid(vector<PT> &p) {
430     int n = p.size(); PT c(0, 0);
431     double sum = 0;
432     for (int i = 0; i < n; i++) sum += cross(p[i], p[(i + 1)
        % n]);
433     double scale = 3.0 * sum;
434     for (int i = 0; i < n; i++) {
435         int j = (i + 1) % n;
436         c = c + (p[i] + p[j]) * cross(p[i], p[j]);
437     }
438     return c / scale;
439 }
440 // 0 if cw, 1 if ccw
441 bool get_direction(vector<PT> &p) {
442     double ans = 0; int n = p.size();
443     for (int i = 0; i < n; i++) ans += cross(p[i], p[(i + 1)
        % n]);
444     if (sign(ans) > 0) return 1;
445     return 0;
446 }
447 // it returns a point such that the sum of distances
448 // from that point to all points in p is minimum
449 // O(n log^2 MX)
450 PT geometric_median(vector<PT> p) {
451     auto tot_dist = [&](PT z) {
452         double res = 0;
453         for (int i = 0; i < p.size(); i++) res += dist(p[i], z
            );
454         return res;
455     };
456     auto findY = [&](double x) {
457         double y1 = -1e5, yr = 1e5;
458         for (int i = 0; i < 60; i++) {
459             double ym1 = y1 + (yr - y1) / 3;
460             double ym2 = yr - (yr - y1) / 3;
461             double d1 = tot_dist(PT(x, ym1));
462             double d2 = tot_dist(PT(x, ym2));
463             if (d1 < d2) yr = ym2;
464             else y1 = ym1;
465         }
466         return pair<double, double> (y1, tot_dist(PT(x, y1)));
467     };
468     double x1 = -1e5, xr = 1e5;
469     for (int i = 0; i < 60; i++) {
470         double xm1 = x1 + (xr - x1) / 3;
471         double xm2 = xr - (xr - x1) / 3;
472         double y1, d1, y2, d2;
473         auto z = findY(xm1); y1 = z.first; d1 = z.second;
474         z = findY(xm2); y2 = z.first; d2 = z.second;
475         if (d1 < d2) xr = xm2;
476         else x1 = xm1;
477     }
478     return {x1, findY(x1).first };
479 }
480 // -1 if strictly inside, 0 if on the polygon, 1 if
    strictly outside
481 // it must be strictly convex, otherwise make it strictly
    convex first
482 int is_point_in_convex(vector<PT> &p, const PT& x) { // O(
    log n)
483     int n = p.size(); assert(n >= 3);
484     int a = orientation(p[0], p[1], x), b = orientation(p
        [0], p[n - 1], x);
485     if (a < 0 || b > 0) return 1;
486     int l = 1, r = n - 1;
487     while (l + 1 < r) {
488         int mid = l + r >> 1;
489         if (orientation(p[0], p[mid], x) >= 0) l = mid;
490         else r = mid;
491     }
492     int k = orientation(p[l], p[r], x);
493     if (k <= 0) return -k;
494     if (l == 1 && a == 0) return 0;
495     if (r == n - 1 && b == 0) return 0;
496     return -1;
497 }
498 bool is_point_on_polygon(vector<PT> &p, const PT& z) {
499     int n = p.size();
500     for (int i = 0; i < n; i++) {
501         if (is_point_on_seg(p[i], p[(i + 1) % n], z)) return
            1;
502     }
503     return 0;
504 }
505 // returns 1e9 if the point is on the polygon
506 int winding_number(vector<PT> &p, const PT& z) { // O(n)
507     if (is_point_on_polygon(p, z)) return 1e9;
508     int n = p.size(), ans = 0;
509     for (int i = 0; i < n; ++i) {
510         int j = (i + 1) % n;
511         bool below = p[i].y < z.y;
512         if (below != (p[j].y < z.y)) {
513             auto orient = orientation(z, p[j], p[i]);
514             if (orient == 0) return 0;
515             if (below == (orient > 0)) ans += below ? 1 : -1;
516         }
517     }
518     return ans;
519 }
520 // -1 if strictly inside, 0 if on the polygon, 1 if
    strictly outside
521 int is_point_in_polygon(vector<PT> &p, const PT& z) { // O(
    n)
522     int k = winding_number(p, z);
523     return k == 1e9 ? 0 : k == 0 ? 1 : -1;
524 }
525
526 double diameter(vector<PT> &p) {
527     int n = (int)p.size();
528     if (n == 1) return 0;
529     if (n == 2) return dist(p[0], p[1]);
530     double ans = 0;
531     int i = 0, j = 1;

```

```

532 while (i < n) {
533     while (cross(p[(i + 1) % n] - p[i], p[(j + 1) % n] -
534             p[j]) >= 0) {
535         ans = max(ans, dist2(p[i], p[j]));
536         j = (j + 1) % n;
537     }
538     ans = max(ans, dist2(p[i], p[j]));
539     i++;
540 }
541 return sqrt(ans);
542 // minimum distance between two parallel lines (non
543 // necessarily axis parallel)
544 // such that the polygon can be put between the lines
545 double width(vector<PT> &p) {
546     int n = (int)p.size();
547     if (n <= 2) return 0;
548     double ans = inf;
549     int i = 0, j = 1;
550     while (i < n) {
551         while (cross(p[(i + 1) % n] - p[i], p[(j + 1) % n] -
552                 p[j]) >= 0) j = (j + 1) % n;
553         ans = min(ans, dist_from_point_to_line(p[i], p[(i +
554             1) % n], p[j]));
555         i++;
556     }
557     return ans;
558 // minimum perimeter
559 double minimum_enclosing_rectangle(vector<PT> &p) {
560     int n = p.size();
561     if (n <= 2) return perimeter(p);
562     int mndot = 0; double tmp = dot(p[1] - p[0], p[0]);
563     for (int i = 1; i < n; i++) {
564         if (dot(p[1] - p[0], p[i]) <= tmp) {
565             tmp = dot(p[1] - p[0], p[i]);
566             mndot = i;
567         }
568     }
569     double ans = inf;
570     int i = 0, j = 1, mxdot = 1;
571     while (i < n) {
572         PT cur = p[(i + 1) % n] - p[i];
573         while (cross(cur, p[(j + 1) % n] - p[j]) >= 0) j = (j
574             + 1) % n;
575         while (dot(p[(mxdot + 1) % n], cur) >= dot(p[mxdot],
576             cur)) mxdot = (mxdot + 1) % n;
577         while (dot(p[(mndot + 1) % n], cur) <= dot(p[mndot],
578             cur)) mndot = (mndot + 1) % n;
579         ans = min(ans, 2.0 * ((dot(p[mxdot], cur) / cur.norm
580             ()) - dot(p[mndot], cur) / cur.norm()) +
581             dist_from_point_to_line(p[i], p[(i + 1) % n], p[
582                 j]));
583         i++;
584     }
585     return ans;
586 // given n points, find the minimum enclosing circle of the
587 // points
588 // call convex_hull() before this for faster solution
589 // expected O(n)
590 circle minimum_enclosing_circle(vector<PT> &p) {
591     random_shuffle(p.begin(), p.end());
592     int n = p.size();
593     circle c(p[0], 0);
594     for (int i = 1; i < n; i++) {
595         if (sign(dist(c.p, p[i]) - c.r) > 0) {
596             c = circle(p[i], 0);
597             for (int j = 0; j < i; j++) {
598                 if (sign(dist(c.p, p[j]) - c.r) > 0) {
599                     c = circle((p[i] + p[j]) / 2, dist(p[i], p[j]
600                         ) / 2);
601                     for (int k = 0; k < j; k++) {
602                         if (sign(dist(c.p, p[k]) - c.r) > 0) {
603                             c = circle(p[i], p[j], p[k]);
604                         }
605                     }
606                 }
607             }
608         }
609     }
610     return c;
611 }
612 pair<PT, int> point_poly_tangent(vector<PT> &p, PT Q, int
613     dir, int l, int r) {
614     while (r - l > 1) {
615         int mid = (l + r) >> 1;
616         bool pvs = orientation(Q, p[mid], p[mid - 1]) != -dir
617             ;
618         bool nxt = orientation(Q, p[mid], p[mid + 1]) != -dir
619             ;
620         if (pvs && nxt) return {p[mid], mid};
621         if (!pvs || !nxt) {
622             auto p1 = point_poly_tangent(p, Q, dir, mid + 1, r
623                 );
624             auto p2 = point_poly_tangent(p, Q, dir, l, mid -
625                 1);
626             return orientation(Q, p1.first, p2.first) == dir ?
627                 p1 : p2;
628         }
629         if (!pvs) {
630             if (orientation(Q, p[mid], p[l]) == dir) r = mid -
631                 1;
632             else if (orientation(Q, p[l], p[r]) == dir) r =
633                 mid - 1;
634             else l = mid + 1;
635         }
636         if (!nxt) {
637             if (orientation(Q, p[mid], p[l]) == dir) l = mid +
638                 1;
639             else if (orientation(Q, p[l], p[r]) == dir) r =
640                 mid - 1;
641             else l = mid + 1;
642         }
643     }
644     pair<PT, int> ret = {p[l], l};
645     for (int i = l + 1; i <= r; i++) ret = orientation(Q,
646         ret.first, p[i]) != dir ? make_pair(p[i], i) : ret;
647     return ret;
648 }
649 // (ccw, cw) tangents from a point that is outside this
650 // convex polygon
651 // returns indexes of the points
652 // ccw means the tangent from Q to that point is in the
653 // same direction as the polygon ccw direction
654 pair<int, int> tangents_from_point_to_polygon(vector<PT> &p
655     , PT Q) {
656     int ccw = point_poly_tangent(p, Q, 1, 0, (int)p.size() -
657         1).second;
658     int cw = point_poly_tangent(p, Q, -1, 0, (int)p.size() -
659         1).second;
660     return make_pair(ccw, cw);
661 }
662 // minimum distance from a point to a convex polygon
663 // it assumes point lie strictly outside the polygon
664 double dist_from_point_to_polygon(vector<PT> &p, PT z) {
665     double ans = inf;
666     int n = p.size();
667     if (n <= 3) {
668         for (int i = 0; i < n; i++) ans = min(ans,
669             dist_from_point_to_seg(p[i], p[(i + 1) % n], z))
670         ;
671     }
672     return ans;
673 }
674 auto [r, l] = tangents_from_point_to_polygon(p, z);
675 if (l > r) r += n;
676 while (l < r) {
677     int mid = (l + r) >> 1;
678     double left = dist2(p[mid % n], z), right = dist2(p[(
679         mid + 1) % n], z);
680     ans = min({ans, left, right});
681     if (left < right) r = mid;
682     else l = mid + 1;
683 }
684 ans = sqrt(ans);
685 ans = min(ans, dist_from_point_to_seg(p[l % n], p[(l +
686     1) % n], z));
687 ans = min(ans, dist_from_point_to_seg(p[l % n], p[(l - 1
688     + n) % n], z));
689 return ans;
690 // minimum distance from convex polygon p to line ab
691 // returns 0 is it intersects with the polygon
692 // top - upper right vertex
693 double dist_from_polygon_to_line(vector<PT> &p, PT a, PT b,
694     int top) { //O(log n)
695     PT orth = (b - a).perp();
696     if (orientation(a, b, p[0]) > 0) orth = (a - b).perp();
697     int id = extreme_vertex(p, orth, top);
698     if (dot(p[id] - a, orth) > 0) return 0.0; //if orth and a
699     // are in the same half of the line, then poly and
700     // line intersects
701     return dist_from_point_to_line(a, b, p[id]); //does not
702     // intersect
703 }
704 // minimum distance from a convex polygon to another convex
705 // polygon
706 // the polygon doesnot overlap or touch
707 // tested in https://toph.co/p/the-wall
708 double dist_from_polygon_to_polygon(vector<PT> &p1, vector<
709     PT> &p2) { // O(n log n)
710     double ans = inf;
711     for (int i = 0; i < p1.size(); i++) {
712         ans = min(ans, dist_from_point_to_polygon(p2, p1[i]))
713         ;
714     }
715     for (int i = 0; i < p2.size(); i++) {
716         ans = min(ans, dist_from_point_to_polygon(p1, p2[i]))
717         ;
718     }
719     return ans;
720 }

```

```

685 }
686 // maximum distance from a convex polygon to another convex
    polygon
687 double maximum_dist_from_polygon_to_polygon(vector<PT> &u,
    vector<PT> &v){ //O(n)
688     int n = (int)u.size(), m = (int)v.size();
689     double ans = 0;
690     if (n < 3 || m < 3) {
691         for (int i = 0; i < n; i++) {
692             for (int j = 0; j < m; j++) ans = max(ans, dist2(u
                [i], v[j]));
693         }
694         return sqrt(ans);
695     }
696     if (u[0].x > v[0].x) swap(n, m), swap(u, v);
697     int i = 0, j = 0, step = n + m + 10;
698     while (j + 1 < m && v[j].x < v[j + 1].x) j++;
699     while (step--){
700         if (cross(u[(i + 1)%n] - u[i], v[(j + 1)%m] - v[j])
                >= 0) j = (j + 1) % m;
701         else i = (i + 1) % n;
702         ans = max(ans, dist2(u[i], v[j]));
703     }
704     return sqrt(ans);
705 }
706 // rotate the polygon such that the (bottom, left)-most
    point is at the first position
707 void reorder_polygon(vector<PT> &p) {
708     int pos = 0;
709     for (int i = 1; i < p.size(); i++) {
710         if (p[i].y < p[pos].y || (sign(p[i].y - p[pos].y) == 0
                && p[i].x < p[pos].x)) pos = i;
711     }
712     rotate(p.begin(), p.begin() + pos, p.end());
713 }
714 // returns the area of the intersection of the circle with
    center c and radius r
715 // and the triangle formed by the points c, a, b
716 double _triangle_circle_intersection(PT c, double r, PT a,
    PT b) {
717     double sd1 = dist2(c, a), sd2 = dist2(c, b);
718     if (sd1 > sd2) swap(a, b), swap(sd1, sd2);
719     double sd = dist2(a, b);
720     double d1 = sqrt1(sd1), d2 = sqrt1(sd2), d = sqrt(sd);
721     double x = abs(sd2 - sd - sd1) / (2 * d);
722     double h = sqrt1(sd1 - x * x);
723     if (r >= d2) return h * d / 2;
724     double area = 0;
725     if (sd + sd1 < sd2) {
726         if (r < d1) area = r * r * (acos(h / d2) - acos(h / d1
                )) / 2;
727         else {
728             area = r * r * (acos(h / d2) - acos(h / r)) / 2;
729             double y = sqrt1(r * r - h * h);
730             area += h * (y - x) / 2;
731         }
732     }
733     else {
734         if (r < h) area = r * r * (acos(h / d2) + acos(h / d1)
                ) / 2;
735         else {
736             area += r * r * (acos(h / d2) - acos(h / r)) / 2;
737             double y = sqrt1(r * r - h * h);
738             area += h * y / 2;
739         }

```

```

740         if (r < d1) {
741             area += r * r * (acos(h / d1) - acos(h / r)) /
                2;
742             area += h * y / 2;
743         }
744         else area += h * x / 2;
745     }
746     return area;
747 }
748 // intersection between a simple polygon and a circle
749 double polygon_circle_intersection(vector<PT> &v, PT p,
    double r) {
750     int n = v.size();
751     double ans = 0.00;
752     PT org = {0, 0};
753     for (int i = 0; i < n; i++) {
754         int x = orientation(p, v[i], v[(i + 1) % n]);
755         if (x == 0) continue;
756         double area = _triangle_circle_intersection(org, r, v
                [i] - p, v[(i + 1) % n] - p);
757         if (x < 0) ans -= area;
758         else ans += area;
759     }
760     return abs(ans);
761 }
762 // find a circle of radius r that contains as many points
    as possible
763 // O(n^2 log n);
764 double maximum_circle_cover(vector<PT> p, double r, circle
    &c) {
765     int n = p.size();
766     int ans = 0;
767     int id = 0; double th = 0;
768     for (int i = 0; i < n; ++i) {
769         // maximum circle cover when the circle goes through
            this point
770         vector<pair<double, int>> events = {{-PI, +1}, {PI,
                -1}};
771         for (int j = 0; j < n; ++j) {
772             if (j == i) continue;
773             double d = dist(p[i], p[j]);
774             if (d > r * 2) continue;
775             double dir = (p[j] - p[i]).arg();
776             double ang = acos(d / 2 / r);
777             double st = dir - ang, ed = dir + ang;
778             if (st > PI) st -= PI * 2;
779             if (st <= -PI) st += PI * 2;
780             if (ed > PI) ed -= PI * 2;
781             if (ed <= -PI) ed += PI * 2;
782             events.push_back({st - eps, +1}); // take care of
                precisions!
783             events.push_back({ed, -1});
784             if (st > ed) {
785                 events.push_back({-PI, +1});
786                 events.push_back({+PI, -1});
787             }
788         }
789         sort(events.begin(), events.end());
790         int cnt = 0;
791         for (auto &&e: events) {
792             cnt += e.second;
793             if (cnt > ans) {
794                 ans = cnt;
795                 id = i; th = e.first;
796             }

```

```

797         }
798     }
799     PT w = PT(p[id].x + r * cos(th), p[id].y + r * sin(th));
800     c = circle(w, r); //best_circle
801     return ans;
802 }
803 // radius of the maximum inscribed circle in a convex
    polygon
804 double maximum_inscribed_circle(vector<PT> p) {
805     int n = p.size();
806     if (n <= 2) return 0;
807     double l = 0, r = 20000;
808     while (r - l > eps) {
809         double mid = (l + r) * 0.5;
810         vector<HP> h;
811         const int L = 1e9;
812         h.push_back(HP(PT(-L, -L), PT(L, -L)));
813         h.push_back(HP(PT(L, -L), PT(L, L)));
814         h.push_back(HP(PT(L, L), PT(-L, L)));
815         h.push_back(HP(PT(-L, L), PT(-L, -L)));
816         for (int i = 0; i < n; i++) {
817             PT z = (p[(i + 1) % n] - p[i]).perp();
818             z = z.truncate(mid);
819             PT y = p[i] + z, q = p[(i + 1) % n] + z;
820             h.push_back(HP(p[i] + z, p[(i + 1) % n] + z));
821         }
822         vector<PT> nw = half_plane_intersection(h);
823         if (!nw.empty()) l = mid;
824         else r = mid;
825     }
826     return l;
827 }

```

Graph

Articulation Point Bridge

```

1 int n; // number of nodes
2 vector<vector<int>> adj; // adjacency list of graph
3 vector<bool> visited;
4 vector<int> tin, low;
5 int timer;
6 void dfs(int v, int p = -1) {
7     visited[v] = true;
8     tin[v] = low[v] = timer++;
9     int children=0;
10    for (int to : adj[v]) {
11        if (to == p) continue;
12        if (visited[to]) {
13            low[v] = min(low[v], tin[to]);
14        } else {
15            dfs(to, v);
16            low[v] = min(low[v], low[to]);
17            if (low[to] >= tin[v] && p != -1)
18                IS_CUTPOINT(v);
19            ++children;
20        }
21    }
22    if (p == -1 && children > 1)
23        IS_CUTPOINT(v);
24 }
25 void find_cutpoints() {
26     timer = 0;

```

```

27     visited.assign(n, false);
28     tin.assign(n, -1);
29     low.assign(n, -1);
30     for (int i = 0; i < n; ++i) {
31         if (!visited[i])
32             dfs(i);
33     }
34 }
35 // finding bridge.
36 void IS_BRIDGE(int v, int to); // some function to process
    the found bridge
37 int n; // number of nodes
38 vector<vector<int>> adj; // adjacency list of graph
39 vector<bool> visited;
40 vector<int> tin, low;
41 int timer;
42 void dfs(int v, int p = -1) {
43     visited[v] = true;
44     tin[v] = low[v] = timer++;
45     bool parent_skipped = false;
46     for (int to : adj[v]) {
47         if (to == p && !parent_skipped) {
48             parent_skipped = true;
49             continue;
50         }
51         if (visited[to]) {
52             low[v] = min(low[v], tin[to]);
53         } else {
54             dfs(to, v);
55             low[v] = min(low[v], low[to]);
56             if (low[to] > tin[v])
57                 IS_BRIDGE(v, to);
58         }
59     }
60 }
61 void find_bridges() {
62     timer = 0;
63     visited.assign(n, false);
64     tin.assign(n, -1);
65     low.assign(n, -1);
66     for (int i = 0; i < n; ++i) {
67         if (!visited[i])
68             dfs(i);
69     }
70 }

```

Bell

```

1 void solve()
2 {
3     vector<int> d(n, INF);
4     d[v] = 0;
5     vector<int> p(n, -1);
6     int x;
7     for (int i = 0; i < n; ++i) {
8         x = -1;
9         for (Edge e : edges)
10             if (d[e.a] < INF)
11                 if (d[e.b] > d[e.a] + e.cost) {
12                     d[e.b] = max(-INF, d[e.a] + e.cost);
13                     p[e.b] = e.a;
14                     x = e.b;
15                 }
16     }
17     if (x == -1)

```

```

18     cout << "No negative cycle from " << v;
19     else {
20         int y = x;
21         for (int i = 0; i < n; ++i)
22             y = p[y];
23         vector<int> path;
24         for (int cur = y;; cur = p[cur]) {
25             path.push_back(cur);
26             if (cur == y && path.size() > 1)
27                 break;
28         }
29         reverse(path.begin(), path.end());
30         cout << "Negative cycle: ";
31         for (int u : path)
32             cout << u << ' ';
33     }
34 }

```

DSU ON Tree

```

1 // DSU on Tree (Sack)
2 // Computes for each node some info about its subtree
    efficiently
3 const int N = 200005;
4 vector<int> tree[N];
5 int col[N], sz[N];
6 bool big[N];
7 int cnt[N]; // frequency of colors
8 int cur_ans = 0; // example answer (can be modified)
9 int answer[N]; // answer for each node
10 // compute subtree sizes
11 void dfs_sz(int u, int p) {
12     sz[u] = 1;
13     for (int v : tree[u]) {
14         if (v == p) continue;
15         dfs_sz(v, u);
16         sz[u] += sz[v];
17     }
18 }
19 // add/remove contribution of subtree
20 void add(int u, int p, int val) {
21     cnt[col[u]] += val;
22     // example logic: maintain max frequency
23     cur_ans = max(cur_ans, cnt[col[u]]);
24     for (int v : tree[u]) {
25         if (v == p || big[v]) continue;
26         add(v, u, val);
27     }
28 }
29 // main DSU dfs
30 void dfs(int u, int p, bool keep) {
31     int mx = -1, heavy = -1;
32     for (int v : tree[u]) {
33         if (v != p && sz[v] > mx) {
34             mx = sz[v];
35             heavy = v;
36         }
37     }
38     // process light children
39     for (int v : tree[u]) {
40         if (v != p && v != heavy) dfs(v, u, false);
41     }
42     // process heavy child
43     if (heavy != -1) {
44         dfs(heavy, u, true); big[heavy] = true;

```

```

45     }
46     // add current node and light children
47     add(u, p, +1); answer[u] = cur_ans;
48     if (heavy != -1) big[heavy] = false;
49     // clear data if not keeping
50     if (!keep) {
51         add(u, p, -1); cur_ans = 0;
52     }
53 }

```

DSU

```

1 void make_set(int v) {
2     parent[v] = v;
3 }
4 int find_set(int v) {
5     if (v == parent[v])
6         return v;
7     return find_set(parent[v]);
8 }
9 void union_sets(int a, int b) {
10     a = find_set(a);
11     b = find_set(b);
12     if (a != b)
13         parent[b] = a;
14 }

```

euler_path_directed

```

1 vector<int> adj[200005];
2 int ptrs[200005];
3 int in_deg[200005], out_deg[200005];
4 vector<int> path; // node sequence
5 vector<int> edges; // edge sequence
6 int to[200005];
7 bool used[200005];
8
9 void dfs(int u) {
10     while(ptrs[u] < adj[u].size()) {
11         int e = adj[u][ptrs[u]++];
12         if (used[e]) continue;
13         used[e] = true;
14         dfs(to[e]);
15         edges.push_back(e);
16     }
17     path.push_back(u);
18 }
19
20
21
22     for (int i = 0; i < m; i++) {
23         int u, v;
24         cin >> u >> v;
25         adj[u].push_back(i);
26         to[i] = v;
27         out_deg[u]++;
28         in_deg[v]++;
29         if (i == 0) s = u;
30     }
31
32     // degree check for Euler path
33     int start = -1, end = -1;
34     bool ok = true;

```

```

35
36     for(int i=0;i<n;i++){
37         int diff = out_deg[i] - in_deg[i];
38         if(diff == 1){
39             if(start != -1) ok = false;
40             start = i;
41         }
42         else if(diff == -1){
43             if(end != -1) ok = false;
44             end = i;
45         }
46         else if(diff != 0){
47             ok = false;
48         }
49     }
50     reverse(path.begin(), path.end());
51     reverse(edges.begin(), edges.end());
52

```

kosaraju

```

1 typedef struct node {
2     int vis;
3     vector<int> adj;
4     vector<int> ulta;
5 } node;
6 node tree[300010];
7 vector<vector<int>> scc(300010);
8 stack<int> st;
9 int scout = 0;
10 void dfs(int u) {
11     tree[u].vis = 1;
12     f(i,tree[u].adj.size()) {
13         int next = tree[u].adj[i];
14         if(tree[next].vis==0) dfs(next);
15     }
16     st.push(u);
17 }
18 void dfs2(int u) {
19     // printf("%d %d\n",u,scout);
20     tree[u].vis = 1;
21     scc[scout-1].push_back(u);
22     f(i,tree[u].ulta.size()) {
23         int next = tree[u].ulta[i];
24         if(tree[next].vis==0) dfs2(next);
25     }
26 }

```

mos on trees

```

1 struct Query { ll id, l, r, bno,k; }; //basic comparator
2 bool cmp(Query &a,Query &b) { if (a.bno != b.bno) {
3     return a.bno < b.bno; } else if (a.bno & 1) { return a
4     .r < b.r; } else { return a.r > b.r; } } ll n,q; ll bs
5     =700;// change is depending on problem Query qry[N];
6     ll a[N],b[N],st[N],en[N]; ll ans[N]; ll tt = -1; vll g
7     [N]; void gg(ll x, ll pr){ //trc(x); tt++; st[x] = tt;
8     a[tt] = x; //trc(tt,a[tt]); go(i,g[x]){ if(i == pr)
9     cu; gg(i,x); } tt++; en[x] = tt; a[tt] = x; } const ll
10    LG = 30; ll tim=0; ll parent[LG][N]; ll tin[N], tout[
11    N], level[N]; void dfs(ll k, ll par, ll lvl) { tin[k
12    ]=++tim; parent[0][k]=par; level[k]=lvl; for(auto it:g
13    [k]) { if(it==par) continue; dfs(it, k, lvl+1); } tout

```

```

[k]=tim; } ll walk(ll u, ll h) { for(ll i=LG-1;i>=0;i
--){ if((h>>i) & 1) u = parent[i][u]; } return u; }
void precompute() { for(ll i=1;i<LG;i++) for(ll j=1;j
<=n;j++) if(parent[i-1][j]) parent[i][j]=parent[i-1][
parent[i-1][j]]; } ll lca(ll u, ll v) { if(level[u]<
level[v]) swap(u,v); ll diff=level[u]-level[v]; for(ll
i=LG-1;i>=0;i--) { if((1<<i) & diff) { u=parent[i][u
]; } } if(u==v) return u; for(ll i=LG-1;i>=0;i--) { if
(parent[i][u] && parent[i][v]!=parent[i][v]) { u=
parent[i][u]; v=parent[i][v]; } } return parent[0][u];
} ll dist(ll u, ll v) { return level[u] + level[v] -
2 * level[lca(u, v)]; } ll lp,rp,cnt; ll freq
[1000001]; ll freq2[1000005]; //unordered_map<ll,ll>
freq,freq2; void add(ll val) { freq[val]++; freq2[b[
val]]++; if(freq[val] == 2){ freq2[b[val]]-=2; // if(
freq2[b[val]] == 0){ // cnt--; // } } else{ // if(
freq2[b[val]] == 1) cnt++; } } void remove(ll val) {
freq[val]--; freq2[b[val]]--; if(freq[val] == 1){
freq2[b[val]]+=2; // if(freq2[b[val]] == 1){ // cnt++;
// } } else{ // if(freq2[b[val]] == 0) cnt--; } } ll
get_answer() { return cnt; } void solve() { cin >> n>>
q; //trc(n,q); mll m; ll dd = 0; for (ll i = 1; i <= n
; i++) { cin >> b[i]; if(!m.count(b[i])){ m[b[i]] = dd
++; } b[i] = m[b[i]]; } f0(n-1){ ipp(x,y); //trc(x,y);
g[x].pb(y); g[y].pb(x); } gg(1,0); dfs(1,0,0);
precompute(); for (ll i = 0; i < q; i++) { ll x,y;
cin >> x >> y; if(level[x]>level[y]){ swap(x,y); } ll
p = lca(x,y); // trc(x,y,p); ll l,r; if(p == x){ l =
st[x]; r = st[y]; qry[i].k = -1; } else{ if(st[x]>st[y
]) swap(x,y); l = en[x]; r = st[y]; qry[i].k = st[p];
// trc(l,r,p,st[p]); // f(j,l,r+1){ // trc(j,a[j]); //
} } qry[i].id = i; qry[i].l = l; qry[i].r = r; qry[i
].bno = l / bs; } // sort the queries sort(qry, qry +
q, cmp); // initialise the data-structure lp = 0; rp =
-1; cnt = 0; // add(a[0]); // answer the queries for
(ll i = 0; i < q; i++) { ll l = qry[i].l; ll r = qry[i
].r; while (rp < r) { add(a[++rp]); } while (lp > l) {
add(a[--lp]); } while (rp > r) { remove(a[rp--]); }
while (lp < l) { remove(a[lp++]); } ans[qry[i].id] =
get_answer(); if(qry[i].k != -1){ // if(freq2[b[a[qry[
i].k]]] == 0){ // ans[qry[i].id]++; // } } } for (ll i
= 0; i < q; i++) { cout << ans[i] << endl; } } //
this require the element to be added on both side l,r
and removed l,r // but we can also use mo's when data
structure allow the removal from the side where we
added like rollback dsu void fun() { solve(); }
int32_t main() { // tasklist // taskkill /f /im a.exe
depressed(); ll tcl = 1; //cin>>tcl; while(tcl--){
fun(); } //cer; return 0; }

```

roll_dsu

```

1 struct DSU { int connected; int sz[N]; pair<int, int> par[N
2 ]; //stores <parent, parity of distance from parent>
3 vector<int> changes; void init(int n) { for(int i=1;i
4 <=n;i++) { par[i]={i, 0}; sz[i]=1; } connected=n; }
5 pair<int, int> getPar(int x) { if(par[x].first==x)
6 return par[x]; pair<int, int> p = getPar(par[x].first)
7 ; p.second ^= par[x].second; return p; } bool unite(
8 int u, int v) //Returns 1 if the graph is no longer
9 bipartite { pair<int, int> x=getPar(u), y=getPar(v);
10 if(x.first==y.first) { if(x.second==y.second) return
11 1; changes.push_back(-1); return 0; } connected--; if(
12 sz[x.first] < sz[y.first]) swap(x, y); par[y.first]=x
13 .first, 1^x.second^y.second; sz[x.first]+=sz[y.first

```

```

}; changes.push_back(y.first); return 0; } void undo()
//Reversed the previous performed operation { if(!
changes.size()) return; if(changes.back()!=-1) { int x
=changes.back(); sz[par[x].first]-=sz[x]; par[x]={x,
0}; } changes.pop_back(); } };

```

Number Theory Math

CRT

```

1 __int128 ExU(__int128 a, __int128 b, __int128 &x, __int128
2 &y) {
3     if(b == 0) {
4         x = 1;
5         y = 0;
6         return a;
7     }
8     __int128 x1, y1;
9     __int128 d = ExU(b, a%b, x1, y1);
10    x = y1;
11    y = x1 - (a/b)*y1;
12    return d;
13 }
14 void solve() {
15     cout << "Case " << ii++ << ": ";
16     int n; cin >> n;
17     vector<int> a(n), m(n);
18     for (int i = 0; i < n; i++) cin >> m[i] >> a[i];
19     int a1 = a[0], m1 = m[0];
20     for (int i = 1; i < n; i++) {
21         __int128 a2 = a[i], m2 = m[i];
22         __int128 x, y;
23         __int128 g = ExU(m1, m2, x, y);
24         if((a1-a2) % g != 0) {
25             cout << "Impossible" << endl;
26             return;
27         }
28         __int128 k = (a1-a2) / g, lcm = (m1/g) * m2;
29         __int128 a = a1 - m1*(k * x);
30         a1 = (a % lcm);
31         if(a1 < 0) a1 += lcm;
32         m1 = lcm;
33     }
34     cout << a1 << endl;
35 }

```

Diophantile

```

1 int gcd(int a, int b, int& x, int& y) {
2     if (b == 0) { x = 1; y = 0; return a;
3     }
4     int x1, y1;
5     int d = gcd(b, a % b, x1, y1);
6     x = y1; y = x1 - y1 * (a / b);
7     return d;
8 }
9
10 bool find_any_solution(int a, int b, int c, int &x0, int &
11 y0, int &g) {
12     g = gcd(abs(a), abs(b), x0, y0);
13     if (c % g) {
14         return false;
15     }

```

```

14     }
15     x0 *= c / g; y0 *= c / g;
16     if (a < 0) x0 = -x0;
17     if (b < 0) y0 = -y0;
18     return true;
19 }
20 void shift_solution(int & x, int & y, int a, int b, int cnt
    ) {
21     x += cnt * b; y -= cnt * a;
22 }
23 int find_all_solutions(int a, int b, int c, int minx, int
    maxx, int miny, int maxy) {
24     int x, y, g;
25     if (!find_any_solution(a, b, c, x, y, g)) return 0;
26     a /= g; b /= g;
27     int sign_a = a > 0 ? +1 : -1;
28     int sign_b = b > 0 ? +1 : -1;
29     shift_solution(x, y, a, b, (minx - x) / b);
30     if (x < minx) shift_solution(x, y, a, b, sign_b);
31     if (x > maxx) return 0;
32     int lx1 = x;
33     shift_solution(x, y, a, b, (maxx - x) / b);
34     if (x > maxx) shift_solution(x, y, a, b, -sign_b);
35     int rx1 = x; shift_solution(x, y, a, b, -(miny - y) / a)
    ;
36     if (y < miny) shift_solution(x, y, a, b, -sign_a);
37     if (y > maxy) return 0;
38     int lx2 = x;
39     shift_solution(x, y, a, b, -(maxy - y) / a);
40     if (y > maxy) shift_solution(x, y, a, b, sign_a);
41     int rx2 = x;
42     if (lx2 > rx2) swap(lx2, rx2);
43     int lx = max(lx1, lx2);
44     int rx = min(rx1, rx2);
45     if (lx > rx) return 0;
46     return (rx - lx) / abs(b) + 1;
47 }

```

FFT

```

1 const int N = 3e5 + 9;
2 const double PI = acos(-1);
3 struct base {
4     double a, b;
5     base(double a = 0, double b = 0) : a(a), b(b) {}
6     const base operator + (const base &c) const
7     { return base(a + c.a, b + c.b); }
8     const base operator - (const base &c) const
9     { return base(a - c.a, b - c.b); }
10    const base operator * (const base &c) const
11    { return base(a * c.a - b * c.b, a * c.b + b * c.a); }
12 };
13 void fft(vector<base> &p, bool inv = 0) {
14     int n = p.size(), i = 0;
15     for(int j = 1; j < n - 1; ++j) {
16         for(int k = n >> 1; k > (i ^= k); k >>= 1);
17         if(j < i) swap(p[i], p[j]);
18     }
19     for(int l = 1, m; (m = l << 1) <= n; l <= m) {
20         double ang = 2 * PI / m;
21         base wn = base(cos(ang), (inv ? 1. : -1.) * sin(ang)), w
            ;
22         for(int i = 0, j, k; i < n; i += m) {
23             for(w = base(1, 0), j = i, k = i + 1; j < k; ++j, w =
                w * wn) {

```

```

24         base t = w * p[j + 1];
25         p[j + 1] = p[j] - t;
26         p[j] = p[j] + t;
27     }
28 }
29 }
30 if(inv) for(int i = 0; i < n; ++i) p[i].a /= n, p[i].b /=
    n;
31 }
32 vector<long long> multiply(vector<int> &a, vector<int> &b)
    {
33     int n = a.size(), m = b.size(), t = n + m - 1, sz = 1;
34     while(sz < t) sz <= 1;
35     vector<base> x(sz), y(sz), z(sz);
36     for(int i = 0; i < sz; ++i) {
37         x[i] = i < (int)a.size() ? base(a[i], 0) : base(0, 0);
38         y[i] = i < (int)b.size() ? base(b[i], 0) : base(0, 0);
39     }
40     fft(x), fft(y);
41     for(int i = 0; i < sz; ++i) z[i] = x[i] * y[i];
42     fft(z, 1);
43     vector<long long> ret(sz);
44     for(int i = 0; i < sz; ++i) ret[i] = (long long) round(z[
        i].a);
45     while((int)ret.size() > 1 && ret.back() == 0) ret.
        pop_back();
46     return ret;
47 }

```

LSieveBigModTernaryS

```

1 int a[], n elements;
2 int mx = 0; int ans = 0;
3 for (int i = 0; i < n; i++) {
4     mx = max(mx+a[i], (int)0);
5     ans = max(ans, mx);
6 }
7 #define sz 1000
8 vector <int> prime;
9 bitset <sz> bs;
10 void sieve() {
11     bs.set(); // all bit = 1..
12     bs[0] = bs[1] = 0;
13     prime.push_back(2);
14     for (int i = 3; i <= sz; i += 2) {
15         if(bs[i]) {
16             for (int j = i*i; j <= sz; j += i)
17                 bs[j] = 0;
18             prime.push_back(i);
19         }
20     }
21 }
22 void solve()
23 {
24     int n = 12;
25     int a[] = {60, 70, 80, 90, 100, 200, 400, 500, 600, 700,
        900, 1000};
26     int r = n-1; int l = 0;
27     // cout << (r-l) / 2;
28     while(l < r) {
29         int c = (r-l) / 3;
30         int m1 = l + c;
31         int m2 = r - c;
32         // cout << a[m1] << ' ' << a[m2] << endl;
33         if(a[m1] < a[m2]) {

```

```

34         r = m2 - 1;
35     } else if(a[m1] > a[m2]) {
36         l = m1 + 1;
37     } else {
38         l = m1 + 1; r = m2 - 1;
39     }
40 }
41 cout << a[l] << endl;
42 }
43 int mod = 1e9 + 7;
44 // linear sieve.
45 vector <int> prime;
46 bool is_composite[MAXN];
47 void sieve (int n) {
48     std::fill (is_composite, is_composite + n, false);
49     for (int i = 2; i < n; ++i) {
50         if (!is_composite[i]) prime.push_back (i);
51         for (int j = 0; j < prime.size () && i * prime[j] < n;
            ++j) {
52             is_composite[i * prime[j]] = true;
53             if (i % prime[j] == 0) break;
54         }
55     }
56 }

```

Miller Rabin

```

1 using u64 = uint64_t;
2 using ul28 = __uint128_t;
3 u64 binpower(u64 base, u64 e, u64 mod) {
4     u64 result = 1;
5     base %= mod;
6     while (e) {
7         if (e & 1)
8             result = (ul28)result * base % mod;
9         base = (ul28)base * base % mod;
10        e >>= 1;
11    }
12    return result;
13 }
14 bool check_composite(u64 n, u64 a, u64 d, int s) {
15     u64 x = binpower(a, d, n);
16     if (x == 1 || x == n - 1)
17         return false;
18     for (int r = 1; r < s; r++) {
19         x = (ul28)x * x % n;
20         if (x == n - 1)
21             return false;
22     }
23     return true;
24 };
25 bool MillerRabin(u64 n, int iter=5) { // returns true if n
    is probably prime, else returns false.
26     if (n < 4)
27         return n == 2 || n == 3;
28     int s = 0;
29     u64 d = n - 1;
30     while ((d & 1) == 0) {
31         d >>= 1;
32         s++;
33     }
34     for (int i = 0; i < iter; i++) {
35         int a = 2 + rand() % (n - 3);
36         if (check_composite(n, a, d, s))
37             return false;

```



```

38 }
39 return true;
40 }
41 bool MillerRabin(u64 n) { // returns true if n is prime,
    else returns false.
42     if (n < 2)
43         return false;
44     int r = 0;
45     u64 d = n - 1;
46     while ((d & 1) == 0) {
47         d >>= 1;
48         r++;
49     }
50     for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
51                 37}) {
52         if (n == a)
53             return true;
54         if (check_composite(n, a, d, r))
55             return false;
56     }
57     return true;

```

NTT

```

1 // Common prime for NTT: 998244353 = 119 * 2^23 + 1
2 // Primitive root: 3
3 const int MOD = 998244353;
4 const int G = 3;
5 // Number Theoretic Transform
6 // on: true for Inverse NTT, false for Forward NTT
7 void ntt(vector<long long>& a, bool invert) {
8     int n = a.size();
9
10    // Bit-reversal permutation
11    for (int i = 1, j = 0; i < n; i++) {
12        int bit = n >> 1;
13        for (; j & bit; bit >>= 1)
14            j ^= bit;
15        j ^= bit;
16        if (i < j) swap(a[i], a[j]);
17    }
18    // Butterfly operations
19    for (int len = 2; len <= n; len <= 1) {
20        long long wlen = power(G, (MOD - 1) / len);
21        if (invert) wlen = modInverse(wlen);
22
23        for (int i = 0; i < n; i += len) {
24            long long w = 1;
25            for (int j = 0; j < len / 2; j++) {
26                long long u = a[i + j], v = (a[i + j + len / 2]
27                    * w) % MOD;
28                a[i + j] = (u + v) < MOD ? (u + v) : (u + v -
29                    MOD);
30                a[i + j + len / 2] = (u - v) >= 0 ? (u - v) : (-
31                    u - v + MOD);
32                w = (w * wlen) % MOD;
33            }
34        }
35    }
36    // Scale results for Inverse NTT
37    if (invert) {
38        long long n_inv = modInverse(n);
39        for (long long& x : a)
40            x = (x * n_inv) % MOD;

```

```

38 }
39 }
40 // Polynomial Multiplication
41 vector<long long> multiply(vector<long long> const& a,
    vector<long long> const& b) {
42     vector<long long> fa(a.begin(), a.end()), fb(b.begin(),
43         b.end());
44     int n = 1;
45     while (n < a.size() + b.size())
46         n <= 1;
47     fa.resize(n);
48     fb.resize(n);
49     ntt(fa, false);
50     ntt(fb, false);
51     for (int i = 0; i < n; i++)
52         fa[i] = (fa[i] * fb[i]) % MOD;
53     ntt(fa, true);
54     // Optional: Resize to actual degree (removes trailing
55         zeros)
56     // while (fa.size() > 1 && fa.back() == 0) fa.pop_back()
57     ;
58     return fa;

```

matexpo

```

1 vector<vector<ll>> matmultiply(vector<vector<ll>>& a,
    vector<vector<ll>>& b) {
2     if(a[0].size() != b.size()) return vector<vector<ll>>(0)
3     ;
4     else {
5         vector<vector<ll>> ans(a.size(), vector<ll>(b[0].size()
6             (), 0));
7         f(i, a.size()) {
8             f(j, b[0].size()) {
9                 f(k, b.size()) ans[i][j] = (ans[i][j] + ((a[i][k] *
10                     b[k][j]) % mod)) % mod;
11             }
12         }
13         return ans;
14     }
15 }
16 vector<vector<ll>> matexpo(vector<vector<ll>>& a, ll exp) {
17     vector<vector<ll>> res(a[0].size(), vector<ll>(a[0].size()
18         (), 0));
19     f(i, a[0].size()) res[i][i] = 1;
20     while(exp > 0) {
21         if(exp & 1) res = matmultiply(res, a);
22         a = matmultiply(a, a);
23         exp /= 2;
24     }
25     return res;
26 }
27 int main() {
28     ll n;
29     scanf("%lld", &n);
30     if(n==0) printf("0\n");
31     else {
32         vector<vector<ll>> arr(2, vector<ll>(2, 1));
33         arr[1][1] = 0;
34         arr = matexpo(arr, n);
35         printf("%lld\n", arr[0][1]);
36     }
37     return 0;

```

```

35 }

```

mobius

```

1 int mob[N];
2 void mobius() {
3     mob[1] = 1;
4     for (int i = 2; i < N; i++) {
5         mob[i]--;
6         for (int j = i + i; j < N; j += i) {
7             mob[j] -= mob[i];
8         }
9     }
10 }
11 bool vis[N];
12 vector<int> d[N];
13 int mul[N];
14 void add(int x, int k) {
15     for (auto y: d[x]) {
16         mul[y] += k;
17     }
18 }
19 int query(int x) {
20     int ans = 0;
21     for (auto y: d[x]) {
22         ans += mul[y] * mob[y];
23     }
24     return ans;
25 }

```

ncr

```

1 // nCr: (n,0)=1, (n,1)=n, (n,r)=(n-1,r)+(n-1,r-1)
2 // Circular permutation: nPr/r
3 ll fact[MAX], inv[MAX], invFact[MAX];
4 // nCr_mod_p_by_lucas_for_small_p._O(LogN)
5 ll Lucas(ll n, ll m) {
6     if (n == 0 && m == 0) return 1ll;
7     ll ni = n % MOD;
8     ll mi = m % MOD;
9     if (mi > ni) return 0;
10    return (Lucas(n / MOD, m / MOD) * nCr(ni, mi)) % MOD;
11 }
12 ll nCr_by_lucas(ll n, ll r) { return Lucas(n, r); }
13 // nCr_mod_M, M_is_not_prime
14 const int N = 142858; // need primes <=M (nCr%M)
15 int spf[N];
16 vector<int> primes;
17 void sieve() {
18     for (int i = 2; i < N; i++) {
19         if (spf[i] == 0) spf[i] = i, primes.push_back(i);
20         int sz = primes.size();
21         for (int j = 0; j < sz && i * primes[j] < N && primes[j]
22             <= spf[i]; j++) spf[i * primes[j]] = primes[j];
23     }
24 }
25 // returns n! % mod, mod = multiple of p
26 // O(mod * log(n))
27 int factmod(ll n, int p, const int mod) {
28     vector<int> f(mod + 1);
29     f[0] = 1 % mod;
30     for (int i = 1; i <= mod; i++) {

```

```

30     if (i % p) f[i] = 1LL * f[i - 1] * i % mod;
31     else f[i] = f[i - 1];
32 }
33 int ans = 1 % mod;
34 while (n > 1) {
35     ans = 1LL * ans * f[n % mod] % mod;
36     ans = 1LL * ans * BigMod(f[mod], n / mod, mod) % mod;
37     n /= p;
38 }
39 return ans;
40 }
41 ll multiplicity(ll n, int p) {
42     ll ans = 0;
43     while (n) {
44         n /= p;
45         ans += n;
46     }
47     return ans;
48 }
49 // C(n, r) modulo p^k
50 // O(p^k log n)
51 int ncr(ll n, ll r, int p, int k) {
52     if (n < r || r < 0) return 0;
53     int mod = 1;
54     for (int i = 0; i < k; i++) { mod *= p; }
55     ll t = multiplicity(n, p) - multiplicity(r, p) -
56         multiplicity(n - r, p);
57     if (t >= k) return 0;
58     int ans = 1LL * factmod(n, p, mod) * inverse(factmod(r,
59         p, mod), mod) % mod * inverse(factmod(n - r, p, mod
60         ), mod) % mod;
61     ans = 1LL * ans * BigMod(p, t, mod) % mod;
62     return ans;
63 }
64 pair<ll, ll> CRT(ll a1, ll m1, ll a2, ll m2) {
65     ll p, q; ll g = e_gcd(m1, m2, p, q);
66     if (a1 % g != a2 % g) return make_pair(0, -1);
67     ll m = m1 / g * m2;
68     p = (p % m + m) % m;
69     q = (q % m + m) % m;
70     return make_pair((p * a2 % m * (m1 / g) % m + q * a1 % m
71         * (m2 / g) % m) % m, m);
72 }
73 // O(m log(n) log(m))
74 int ncr(ll n, ll r, int m) {
75     if (n < r || r < 0) return 0;
76     pair<ll, ll> ans({0, 1});
77     while (m > 1) {
78         int p = spf[m], k = 0, cur = 1;
79         while (m % p == 0) {
80             m /= p; cur *= p; ++k;
81         }
82         ans = CRT(ans.first, ans.second, ncr(n, r, p, k), cur
83             );
84     }
85     return ans.first;
86 }

```

stirling_number

```

1 // S(n, k) = (1/k!) * sum_{i=0..k} (-1)^i * C(k, i) * (k-i)^n
2 int main() {
3     int n, k; cin >> n >> k;
4     vector<ll> fact(k+1, 1), invfact(k+1, 1);
5     ll ans = 0;

```

```

6     for (int i = 0; i <= k; i++) {
7         ll term = fact[k] * invfact[i] % MOD * invfact[k-i] %
8             MOD;
9         term = term * power(k - i, n) % MOD;
10        if (i & 1) ans = (ans - term + MOD) % MOD;
11        else ans = (ans + term) % MOD;
12    }
13    ans = ans * modinv(fact[k]) % MOD;
14    cout << ans << "\n";
15 }

```

totient

```

1 int phi(int n) {
2     int result = n;
3     for (int i = 2; i * i <= n; i++) {
4         if (n % i == 0) {
5             while (n % i == 0) n /= i;
6             result -= result / i;
7         }
8     }
9     if (n > 1) result -= result / n;
10    return result;
11 }
12 void phi_1_to_n(int n) {
13     vector<int> phi(n + 1);
14     for (int i = 0; i <= n; i++) phi[i] = i;
15     for (int i = 2; i <= n; i++) {
16         if (phi[i] == i) {
17             for (int j = i; j <= n; j += i) phi[j] -= phi[j] /
18                 i;
19         }
20     }
21 }
22 void phi_1_to_n(int n) {
23     vector<int> phi(n + 1);
24     phi[0] = 0; phi[1] = 1;
25     for (int i = 2; i <= n; i++) phi[i] = i - 1;
26
27     for (int i = 2; i <= n; i++)
28         for (int j = 2 * i; j <= n; j += i) phi[j] -= phi[i];
29 }

```

String

AhoCorasick

```

1 typedef struct AC {
2     static const int K = 26;
3     struct node {
4         int nxt[K], link, leaf, par;
5         vector<int> ind;
6         char pch;
7         node(int par = -1, char pch = '$') : par(par), pch(
8             pch) {
9             fill(begin(nxt), end(nxt), -1);
10            leaf = false;
11            link = -1;
12        }
13    };
14    vector<node> t;
15    vector<vector<int>> ad;

```

```

16 vector<int> st, en, mp;
17 vector<int> cnt;
18 int dt = 0;
19 AC() {
20     mp = vector<int> (1<<8, 0);
21     for (char ch = 'a'; ch <= 'z'; ++ch) mp[ch] = ch - 'a
22         ';
23     t.resize(1);
24     cnt.resize(1);
25 }
26 int add(string str, int i) {
27     int ptr = 0;
28     for (auto ch : str) {
29         if (t[ptr].nxt[mp[ch]] < 0) {
30             t[ptr].nxt[mp[ch]] = t.size();
31             t.push_back(node(ptr, ch));
32             cnt.push_back(0);
33         }
34         ptr = t[ptr].nxt[mp[ch]];
35     }
36     t[ptr].leaf = true;
37     t[ptr].ind.push_back(i);
38     return ptr;
39 }
40 int get_link(int v) {
41     if (t[v].link == -1) {
42         t[v].link = (!v || !t[v].par) ? 0 : go(get_link(t[
43             v].par), t[v].pch);
44     }
45     return t[v].link;
46 }
47 int go(int v, char ch) {
48     if (t[v].nxt[mp[ch]] < 0) {
49         t[v].nxt[mp[ch]] = !v ? 0 : go(get_link(v), ch);
50     }
51     return t[v].nxt[mp[ch]];
52 }
53 void dfs(int v) {
54     st[v] = ++dt;
55     for (int u : ad[v]) {
56         dfs(u);
57         cnt[v] += cnt[u];
58     }
59     en[v] = dt;
60 }
61 void calc() {
62     int k = t.size();
63     st.resize(k);
64     en.resize(k);
65     ad.resize(k);
66     for (int i = 1; i < k; i++)
67         ad[get_link(i)].push_back(i);
68     dfs(0);
69 }
70 } cc;
71 int cs = 1;
72 void solve() {
73     cc bc;
74     int n; cin >> n;
75     vector<int> res(n + 1, 0);
76     string text;
77     cin >> text;
78     string p;
79     for (int i = 1; i <= n; i++) {
80         cin >> p;

```

```

78     bc.add(p, i);
79 }
80 int vertex = 0;
81 for (int i = 0; i < text.length(); i++) {
82     vertex = bc.go(vertex, text[i]);
83     bc.cnt[vertex]++;
84 }
85 bc.calc();
86 for (int i = 1; i < bc.t.size(); i++) {
87     for (int &x : bc.t[i].ind) res[x] += bc.cnt[i];
88 }
89 cout << "Case " << cs++ << ":\n";
90 for (int i = 1; i <= n; i++) cout << res[i] << "\n";
91 }

```

Manacher

```

1 struct Manacher {
2     vector<int> p[2];
3     // p[1][i] = (max odd length palindrome centered at i) /
4     // 2 [floor division]
5     // p[0][i] = same for even, it considers the right center
6     // e.g. for s = "abbabba", p[1][3] = 3, p[0][2] = 2
7     Manacher(string s) {
8         int n = s.size();
9         p[0].resize(n + 1);
10        p[1].resize(n);
11        for (int z = 0; z < 2; z++) {
12            for (int i = 0, l = 0, r = 0; i < n; i++) {
13                int t = r - i + !z;
14                if (i < r) p[z][i] = min(t, p[z][l + t]);
15                int L = i - p[z][i], R = i + p[z][i] - !z;
16                while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
17                    p[z][i]++, L--, R++;
18                if (R > r) l = L, r = R;
19            }
20        }
21        bool is_palindrome(int l, int r) {
22            int mid = (l + r + 1) / 2, len = r - l + 1;
23            return 2 * p[len % 2][mid] + len % 2 >= len;
24        }
25    };
26    int32_t main() {
27        string s; cin >> s;
28        Manacher M(s);
29        int n = s.size();
30        for (int i = 0; i < n; i++) {
31            cout << 2 * M.p[1][i] + 1 << ' ';
32            if (i + 1 < n) cout << 2 * M.p[0][i + 1] << ' ';
33        }
34        cout << '\n';
35        return 0;
36    }

```

Suffix Automata

```

1 // Suffix Automaton (SAM)
2 // Handles all substrings of a string in linear time
3 struct State {
4     int next[26];
5     int link, len;
6     State() {

```

```

7         memset(next, -1, sizeof next);
8         link = -1;
9         len = 0;
10    }
11 };
12 vector<State> st;
13 int last;
14 // initialize
15 void sa_init() {
16     st.clear(); st.push_back(State()); last = 0;
17 }
18 // extend by character c
19 void sa_extend(char ch) {
20     int c = ch - 'a';
21     int cur = st.size();
22     st.push_back(State());
23     st[cur].len = st[last].len + 1;
24     int p = last;
25     while (p != -1 && st[p].next[c] == -1) {
26         st[p].next[c] = cur;
27         p = st[p].link;
28     }
29     if (p == -1) {
30         st[cur].link = 0;
31     } else {
32         int q = st[p].next[c];
33         if (st[p].len + 1 == st[q].len) {
34             st[cur].link = q;
35         } else {
36             int clone = st.size();
37             st.push_back(st[q]);
38             st[clone].len = st[p].len + 1;
39             while (p != -1 && st[p].next[c] == q) {
40                 st[p].next[c] = clone; p = st[p].link;
41             }
42             st[q].link = st[cur].link = clone;
43         }
44     }
45     last = cur;
46 }
47 // count distinct substrings
48 long long count_distinct() {
49     long long ans = 0;
50     for (auto &s : st)
51         ans += s.len - (s.link == -1 ? 0 : st[s.link].len);
52     return ans;
53 }

```

palindromitree

```

1 const int MAXN = 105000;
2 struct node {
3     int next[26];
4     int len;
5     int sufflink;
6     int num;
7 };
8 int len;
9 char s[MAXN];
10 node tree[MAXN];
11 int num; // node 1 - root with len -1, node 2 - root with
12 // len 0
13 int suff; // max suffix palindrome
14 long long ans;
15 bool addLetter(int pos) {

```

```

15     int cur = suff, curlen = 0;
16     int let = s[pos] - 'a';
17     while (true) {
18         curlen = tree[cur].len;
19         if (pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s
20             [pos])
21             break;
22         cur = tree[cur].sufflink;
23     }
24     if (tree[cur].next[let]) {
25         suff = tree[cur].next[let];
26         return false;
27     }
28     num++;
29     suff = num;
30     tree[num].len = tree[cur].len + 2;
31     tree[cur].next[let] = num;
32     if (tree[num].len == 1) {
33         tree[num].sufflink = 2;
34         tree[num].num = 1;
35         return true;
36     }
37     while (true) {
38         cur = tree[cur].sufflink;
39         curlen = tree[cur].len;
40         if (pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s
41             [pos]) {
42             tree[num].sufflink = tree[cur].next[let];
43             break;
44         }
45     }
46     tree[num].num = 1 + tree[tree[num].sufflink].num;
47     return true;
48 }
49 void initTree() {
50     num = 2; suff = 2;
51     tree[1].len = -1; tree[1].sufflink = 1;
52     tree[2].len = 0; tree[2].sufflink = 1;
53 }
54 int main() {
55     //assert(freopen("input.txt", "r", stdin));
56     //assert(freopen("output.txt", "w", stdout));
57     gets(s); len = strlen(s);
58     initTree();
59     for (int i = 0; i < len; i++) {
60         addLetter(i);
61         ans += tree[suff].num;
62     }
63 }

```

suffixArray

```

1 struct SuffixArray {
2     int n; string s;
3     vector<int> sa, rankv, tmp, lcp;
4     SuffixArray(const string &str) {
5         s = str; n = s.size(); sa.resize(n); rankv.resize(n);
6         tmp.resize(n); build(); build_lcp();
7     }
8     void counting_sort(int k) {
9         int maxi = max(256ll, (long long)n) + 5;
10        vector<int> cnt(maxi, 0), sa2(n);
11        for (int i = 0; i < n; i++) {
12            int key = (i + k < n) ? rankv[i + k] + 1 : 0;
13            cnt[key]++;

```

```

13     }
14     for (int i = 1; i < maxi; i++) cnt[i] += cnt[i - 1];
15     for (int i = n - 1; i >= 0; i--) {
16         int key = (sa[i] + k < n) ? rankv[sa[i] + k] + 1 :
17             0;
18         sa2[--cnt[key]] = sa[i];
19     }
20     sa.swap(sa2);
21 }
22 void build() {
23     for (int i = 0; i < n; i++) {
24         sa[i] = i; rankv[i] = (unsigned char)s[i];
25     }
26     for (int k = 1; k < n; k <= 1) {
27         counting_sort(k);
28         counting_sort(0);
29         tmp[sa[0]] = 0;
30         int r = 0;
31         for (int i = 1; i < n; i++) {
32             int cur1 = rankv[sa[i]], cur2 = (sa[i] + k < n
33                 ? rankv[sa[i] + k] : -1);
34             int prev1 = rankv[sa[i - 1]], prev2 = (sa[i -
35                 1] + k < n ? rankv[sa[i - 1] + k] : -1);
36             if (cur1 != prev1 || cur2 != prev2) r++;
37             tmp[sa[i]] = r;
38         }
39         rankv.swap(tmp);
40         if (r == n - 1) break;
41     }
42 }
43 void build_lcp() {
44     lcp.assign(max(0, n - 1), 0);
45     vector<int> inv(n);
46     for (int i = 0; i < n; ++i) inv[sa[i]] = i;
47     int k = 0;
48     for (int i = 0; i < n; ++i) {
49         if (inv[i] == n - 1) { k = 0; continue; }
50         int j = sa[inv[i] + 1];
51         while (i + k < n && j + k < n && s[i + k] == s[j +
52             k]) ++k;
53         lcp[inv[i]] = k;
54         if (k) --k;
55     }
56 }
57 };
58 void solve(const string &s, const string &t) {
59     // build local combined string so we don't mutate inputs
60     int n = s.size();
61     string combined = s + '$' + t;
62     int N = combined.size();
63     SuffixArray sa(combined);
64 }

```

```

11     }
12     return pi;
13 }
14 vector<int> z_function(string pattern, string text) {
15     string s = pattern + "$" + text;
16     int n = size(s);
17     vector<int> z(n);
18     for (int i = 1, l = 0, r = 0; i < n; ++i) {
19         if (i <= r)
20             z[i] = min (r - i + 1, z[i - l]);
21         while (i + z[i] < n && s[z[i]] == s[i + z[i]])
22             ++z[i];
23         if (i + z[i] - 1 > r)
24             l = i, r = i + z[i] - 1;
25     }
26     return z;
27 }

```

z kmp

```

1 vector<int> prefix_function(string s) {
2     int n = (int)s.length();
3     vector<int> pi(n);
4     for (int i = 1; i < n; i++) {
5         int j = pi[i-1];
6         while (j > 0 && s[i] != s[j])
7             j = pi[j-1];
8         if (s[i] == s[j])
9             j++;
10        pi[i] = j;

```